

Prague University of Economics and Business

Faculty of Informatics and Statistics



GPU Acceleration for Inductive Logic Rule Mining in Graphs

MASTER THESIS

Study program: Knowledge Engineering and Web Technologies

Author: Bc. Vojtech Naar

Master thesis supervisor: Ing. Václav Zeman, Ph.D.

Prague, May 2026

Declaration on the Use of Artificial Intelligence

I confirm that artificial intelligence tools were used in the preparation of the submitted work. Specifically, they were used in the following ways:

- improvement of grammar and language style,
- searching for and summarising literature,
- editing and expanding text sections,
- generating parts of source code.

Each use of any of the above methods is individually documented in the appendix of the work, where a more detailed explanation is provided specifying which tool was used in which part of the work, and the author's own contribution is highlighted.

I confirm that:

- my work is original and was processed in accordance with ethical standards, in particular with the Code of Ethics of the Prague University of Economics and Business, that all information sources used in the work are cited, and that the generated content has been verified by me,
- I accept full responsibility for the entire content of the final thesis.

Acknowledgement

I would like to thank my thesis supervisor Ing. Václav Zeman, Ph.D. for his guidance throughout this project. I am also grateful to the Faculty of Informatics and Statistics at Prague University of Economics and Business for providing access to the VSE JupyterHub environment with the NVIDIA H200 NVL GPU used in all benchmarks. Special thanks to the authors of RDRules, whose open-source framework served as the baseline and motivation for this work.

Abstract

This master thesis focuses on accelerating support and confidence computation in inductive logic rule mining over large RDF knowledge graphs. The work builds on AMIE-style Horn rule mining and the RDFRules framework, where candidate rules are evaluated by repeatedly executing join-like graph pattern matching operations over large triple sets. The thesis investigates whether this counting phase can be accelerated using native C++ implementations and NVIDIA CUDA GPU execution while preserving the original rule semantics.

The practical contribution consists of a faithful C++ rewrite of the RDFRules counting logic, a cache-oriented CPU implementation based on CSR indexes, and a CUDA implementation that stores graph indexes in GPU memory and evaluates rules using massively parallel DFS kernels. The implementations are integrated with the Scala RDFRules framework through a shared native library and Java Native Access. Experimental evaluation on synthetic, DBpedia, and biomedical datasets shows that GPU acceleration is strongest when rule evaluation produces large body-size search spaces, while CPU-side CSR optimization is highly dependent on graph topology and may regress on high-fanout predicates.

Keywords

Logic rule, graphs mining, GPU, cuda, parallelism.

Table of Contents

1	Theoretical Background of the Thesis	10
1.1	Parallel Hardware Background and GPU Computing	10
1.1.1	CPU vs GPU execution model.....	10
1.1.2	CUDA programming model	14
1.1.3	GPU memory hierarchy	17
1.1.4	Applicability of GPUs to Different Computational Workloads	18
1.2	Graph Data and Rule Mining Background	19
1.2.1	Knowledge graphs and RDF-style data	19
1.2.2	Rule mining and Inductive Logic Programming Basics	20
1.2.3	Computational bottlenecks on CPU	22
1.3	Key Software Libraries and Runtime Dependencies	24
1.3.1	NVIDIA CUDA Toolkit and NVCC	25
1.3.2	Thrust	25
1.3.3	cuSPARSE	25
1.3.4	Java Native Access	26
1.4	Related work	26
2	Implementation and Optimization of Support Counting	28
2.1	Scala Baseline: RDRules Library	28
2.1.1	Role of RDRules in the Thesis	28
2.1.2	Data Loading and Index Construction	29
2.1.3	Rule Representation and Counting Semantics	29
2.1.4	Performance Characteristics	29
2.2	Faithful C++ Rewrite	30
2.2.1	Design Goal	30
2.2.2	Graph Storage	30
2.2.3	Support and Body-Size Counting	30
2.2.4	Correctness Role	31
2.2.5	Limitations	31
2.3	Optimized Non-Parallel CPU Implementation	31
2.3.1	CSR Graph Layout.....	31

2.3.2	Stack-Local DFS Environment	32
2.3.3	Bitmask Atom Tracking	32
2.3.4	Template-Specialized DFS	32
2.3.5	Expected Advantages and Risks	32
2.4	GPU Implementation Trajectory	33
2.4.1	GPU Attempt 1: Parallel DFS with Binary-Search Lookup	33
2.4.2	GPU Attempt 2: Sparse Matrix Formulation	34
2.5	Final GPU Implementation: GPU_03	36
2.5.1	Design Rationale	36
2.5.2	Support Counting on GPU	36
2.5.3	Body-Size and Confidence Counting on GPU	37
2.5.4	CUDA-Specific Techniques	38
2.5.5	Trade-Offs	39
3	Integration with the Scala RDRules Framework	40
3.1	Architecture Overview	40
3.2	Compiling the Native Library	41
3.3	Loading the Library from Scala	41
3.4	Data Flow and Output Layout	42
3.5	CPU Fallback Path	42
4	Experimental Evaluation	43
4.1	Experimental Setup and Benchmark Methodology	43
4.2	Datasets and Rule Sets	45
4.3	Scala Baseline versus Faithful C++ Rewrite	46
4.4	Synthetic Stress Test: gpu-test-100k	47
4.5	DBpedia Dataset: 10,000 Rules	48
4.6	Biomedical Dataset: original_train	49
4.7	Memory Usage	50
4.8	Cross-Dataset Discussion	51
4.9	Implementation Selection Guidelines	53
5	Conclusion	54
	Sources	56
	Schema Sources	58

Appendix.....	59
Rewritten C++ — Baseline Implementation	59
Optimized CPU Implementation.....	59
GPU CUDA Implementation.....	60
Benchmarking and Evaluation Scripts.....	60
Data Generation Scripts	60

List of Tables

Table 1 - C API functions exported by libgpu_rules.so.	39
Table 2 - Binary output layout returned by the native GPU backend.	39
Table 3 - Benchmark machine hardware specification.	40
Table 4 - Datasets used in the experimental evaluation.	40
Table 5 - Support counting on the biomedical graph, 931 rules.	41
Table 6 - Wall times on the synthetic 100k dataset.	42
Table 7 - Rule-computation time on the synthetic 100k dataset.	42
Table 8 - Selected per-rule confidence times on the synthetic dataset.	42
Table 9 - DBpedia benchmark results.	43
Table 10 - DBpedia support and confidence phase breakdown.	43
Table 11 - DBpedia confidence times by body-atom count.	44
Table 12 - Biomedical 50-rule benchmark results.	44
Table 13 - Outlier rules on the biomedical dataset.	45
Table 14 - Biomedical confidence time by body-atom count.	45
Table 15 - Peak memory usage across datasets.	46
Table 16 - Cross-dataset rule-computation summary.	46
Table 17 - Recommended implementation by workload scenario.	47

Introduction

This thesis addresses the problem of computational efficiency in inductive logic rule mining over large graph-structured data. The motivation for this work stems from the rapidly increasing size of real-world knowledge graphs and the high computational complexity of logic-based rule mining algorithms, which often limits their scalability and practical usability. The main objective of this thesis is to investigate whether selected components of an inductive logic rule mining algorithm can be accelerated using GPU parallelism, with particular emphasis on support counting and other computationally intensive graph operations.

The thesis is divided into a theoretical and a practical part. The theoretical part provides an overview of inductive logic rule mining in graphs, with a focus on identifying their computationally expensive components. It further discusses the architectural differences between CPUs and GPUs, explains the principles of GPU parallelism, and analyzes scenarios in which GPU acceleration is effective compared to traditional CPU-based computation. The practical part focuses on the implementation of selected parts of the rule mining algorithm using CUDA and GPU acceleration. The proposed GPU-based solutions are experimentally evaluated on graph datasets to assess their performance, scalability, and potential speedup over CPU implementations. Following this introduction, the thesis is structured into numbered chapters that elaborate on the theoretical background, methodology, implementation details, and experimental results.

1 Theoretical Background of the Thesis

1.1 Parallel Hardware Background and GPU Computing

1.1.1 CPU vs GPU execution model

Modern computing systems employ both central processing units (CPUs) and graphics processing units (GPUs), which differ significantly in their execution models and performance characteristics. These architectural differences reflect distinct design goals: CPUs prioritize low-latency execution and efficient handling of complex control flow, while GPUs are optimized for data-parallel workloads in which large numbers of similar operations can be performed concurrently. As discussed later in Section 1.2.3, the evaluation of candidate rules in knowledge graph mining consists of repeated join and counting operations over large triple sets. These operations can often be decomposed into independent computations over disjoint bindings, suggesting that they may be amenable to parallel execution on GPU architectures.

The CPU serves as the primary control and execution component of a computer system. Its main purpose is to coordinate program execution by fetching instructions from memory, interpreting them, and performing the required operations on data. A typical CPU consists of several key components, including processing cores, arithmetic and logic units responsible for computation, control logic for instruction sequencing, and registers used for fast temporary storage. Modern CPUs are also tightly integrated with a cache hierarchy, which reduces memory access latency and improves overall system performance. Together, these components enable the CPU to efficiently execute a wide range of general-purpose workloads while maintaining precise control over program flow and system state.

A typical CPU is composed of the following core components, each responsible for a specific aspect of instruction processing:

Processing cores – Execute program instructions and contain the main execution logic of the CPU.

Arithmetic Logic Unit (ALU) – Performs arithmetic operations, logical operations, and comparisons.

Control unit – Manages instruction flow, decoding, and control signal generation.

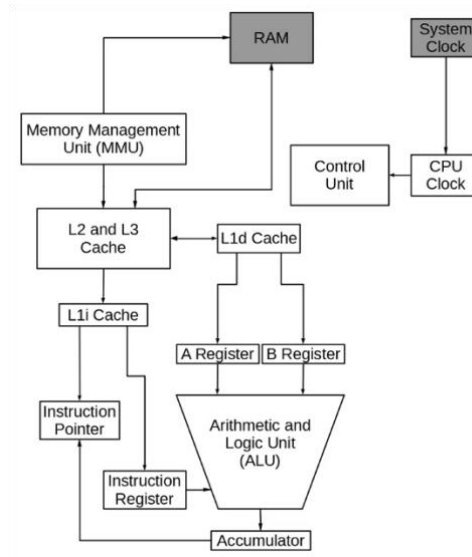
Registers – Small, high-speed storage locations for operands, intermediate results, and control information such as the program counter.

Cache memory – A hierarchy of fast memories (L1, L2, L3) that reduce access latency to main memory.

Memory interface – Handles communication between the CPU and main memory.

Modern CPUs are general-purpose processors that execute programs as sequences of instructions defined by an instruction set architecture (ISA). The ISA specifies the instructions and resources visible to software and forms the interface between hardware and software, while the internal CPU design determines how efficiently these instructions are processed. In contemporary computer systems, computer architecture cannot be understood as ISA design alone. Practical CPU design also includes the processor organization, or microarchitecture, which encompasses aspects such as the memory system, instruction pipelines, and the implementation of arithmetic, control, and data movement operations, as well as the underlying hardware realization. Consequently, processors that implement the same ISA may still differ significantly in performance due to variations in pipeline structure, cache hierarchy, and other microarchitectural decisions (Hennessy and Patterson, 2012).

At the operational level, CPUs execute programs by repeatedly performing an instruction processing cycle commonly referred to as the **fetch–decode–execute** cycle. During the fetch phase, the CPU retrieves the next instruction from memory using the program counter, typically via the cache hierarchy. In the decode phase, the control logic interprets the instruction, determines the required operation, and identifies the necessary operands and execution units. The execute phase then performs the specified operation, such as an arithmetic computation, a logical comparison, a memory access, or a control flow update, using the appropriate functional units. Finally, the results are written back to registers or memory, and the program counter is updated to point to the next instruction. Modern CPUs overlap multiple instances of this cycle through pipelining and other microarchitectural optimizations, allowing several instructions to be processed concurrently and thereby improving overall performance.



Schema 1 — CPU Architecture Diagram (Red Hat, Inc., n.d.)

GPUs are specialized processors designed to efficiently execute workloads that exhibit a high degree of parallelism. While originally developed to accelerate graphics rendering, modern GPUs have evolved into general-purpose computing devices capable of supporting a wide range of non-graphics applications. Their architecture is particularly well suited for computations in which the same operations are applied repeatedly to large datasets, making GPUs an important platform for scientific computing, data analytics, and machine learning.

A modern GPU consists of several key components that together support massively parallel execution:

Streaming Multiprocessors (SMs) – The primary execution units of the GPU, each containing multiple simple processing cores and responsible for executing groups of threads in parallel.

Processing cores (CUDA cores / stream processors) – Lightweight arithmetic units optimized for floating-point and integer computations, designed to execute a single instruction across many threads.

Thread scheduler – Manages the execution of large numbers of threads and rapidly switches between them to hide memory and execution latencies.

Memory hierarchy – Includes registers, shared memory, caches, and high-bandwidth global memory, providing different trade-offs between access speed and capacity.

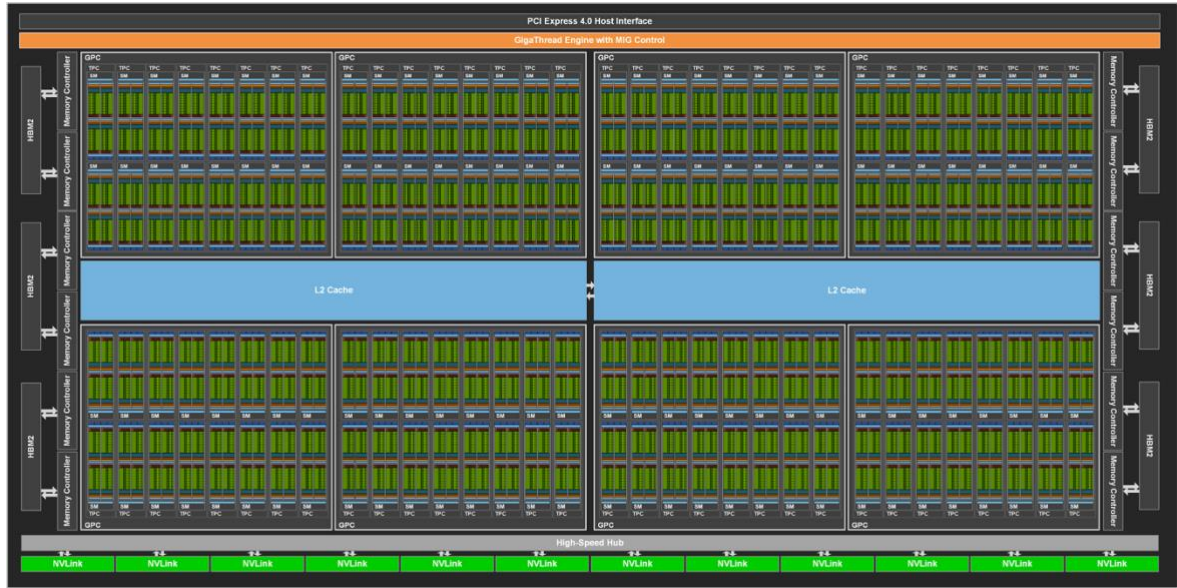
Global memory interface – Connects the GPU to off-chip high-bandwidth memory, enabling fast access to large datasets required by parallel workloads.

Host interface – Facilitates communication between the CPU and GPU, allowing data transfers and coordination between sequential and parallel parts of an application.

The increasing relevance of GPUs in general-purpose computing is closely related to changes in processor design trends over the past two decades. As improvements in single-core CPU performance became constrained by power consumption and heat dissipation limits, further performance gains increasingly depended on exploiting parallel execution. GPUs address this challenge by integrating a large number of relatively simple processing cores that collectively execute thousands of lightweight threads. This design enables GPUs to achieve substantially higher aggregate computational throughput than traditional processors for workloads that expose sufficient parallelism (Kirk and Hwu, 2016).

From an architectural perspective, GPUs follow a throughput-oriented design philosophy that prioritizes arithmetic density and memory bandwidth over complex control mechanisms. Rather than minimizing the latency of individual instructions, GPUs rely on massive concurrency to tolerate long-latency operations such as memory accesses. When some threads are stalled, others can continue execution, allowing the hardware to keep execution units highly utilized. This approach makes GPUs particularly effective for data-parallel workloads with regular control flow, while reducing their efficiency for tasks dominated by sequential execution or irregular branching (Kirk and Hwu, 2016).

A modern GPU is organized as an array of **Streaming Multiprocessors (SMs)**, which represent the primary units of parallel execution. Each SM contains multiple lightweight processing cores, a hardware scheduler, and fast on-chip resources such as registers and shared memory. Threads are executed in groups according to the single-instruction, multiple-thread (SIMT) model, enabling efficient execution of identical instructions across many data elements. By scaling both the number of SMs and the resources available within each SM, GPUs can support extremely high levels of parallelism and achieve the high computational throughput illustrated in the following schematics.



Schema 2 — GA100 GPU with 128 SMs (Krashinsky et al., 2020)



Schema 3 — GA100 SM Architecture (Krashinsky et al., 2020)

1.1.2 CUDA programming model

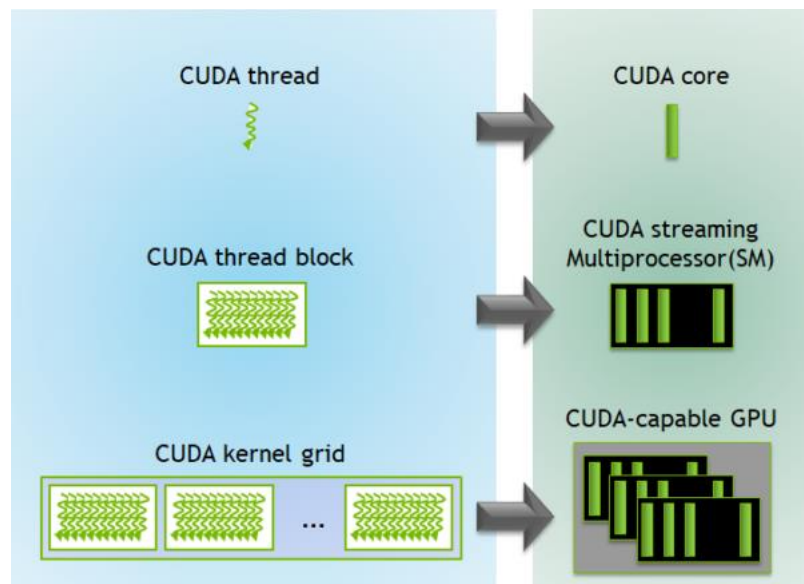
The CUDA programming model was introduced by NVIDIA to enable general-purpose computation on graphics processing units by exposing their parallel execution capabilities through a programming interface accessible to software developers. Before the introduction of CUDA, the use of GPUs for non-graphics tasks required programming through graphics-oriented APIs, which significantly limited flexibility and usability. CUDA removed this

limitation by allowing developers to express parallel computation directly using familiar programming languages such as C and C++.

The motivation for CUDA is closely tied to long-term trends in processor architecture. As performance improvements from increasing CPU clock frequencies became constrained by power consumption and heat dissipation, further gains increasingly depended on exploiting parallel execution. GPUs, designed around many-core architectures and high memory bandwidth, emerged as an effective platform for accelerating data-parallel workloads. CUDA was created to make this hardware accessible beyond graphics applications by providing explicit control over parallel execution and memory usage (Kirk and Hwu, 2013).

CUDA follows a heterogeneous execution model in which computation is divided between the CPU and the GPU. The CPU is responsible for overall application control and coordination, while the GPU executes computationally intensive parts of the program in parallel. This model allows applications to combine sequential control logic with massively parallel execution in a single program and reflects the complementary strengths of both processors.

Within this model, GPU computation is expressed in the form of kernels, which are launched by the CPU and executed concurrently by a large number of GPU threads. Each kernel launch defines how parallel work is structured and scheduled on the device. Because the CPU and GPU maintain separate memory spaces, data must be transferred explicitly between host and device memory, making data movement an important factor in overall performance (Cheng et al., 2014).



Schema 4 — Kernel execution on GPU (NVIDIA Corporation, 2020)

Parallel execution in CUDA is structured using a hierarchical thread model that enables scalability across GPU architectures. A kernel is executed by a grid of threads, where the grid represents the complete set of threads associated with a kernel invocation. The grid is subdivided into thread blocks, which are independent units of execution that can be scheduled on different streaming multiprocessors. This abstraction allows CUDA programs to express parallelism independently of the underlying hardware configuration (Kirk and Hwu, 2013).

Threads within a block can cooperate through fast on-chip shared memory and synchronize their execution using barrier primitives. Threads belonging to different blocks execute independently and cannot directly synchronize during kernel execution. This design ensures scalability and flexibility, as blocks may be scheduled and executed in any order without affecting program correctness.

CUDA GPUs execute threads according to the single-instruction, multiple-thread (SIMT) execution model. In this model, threads are grouped into fixed-size units called warps, which are scheduled and executed together by the hardware. A single instruction is issued for an entire warp and applied simultaneously to all threads within it, enabling efficient execution of data-parallel workloads (Kirk and Hwu, 2013).

Although threads within a warp share an instruction stream, each thread maintains its own execution state. When threads within a warp follow different execution paths due to conditional branching, execution must be serialized for each path, resulting in reduced efficiency. This phenomenon, known as warp divergence, has a significant impact on performance and influences how algorithms should be structured for GPU execution (Cheng et al., 2014).

The CUDA programming model also defines a hierarchical memory system designed to balance access latency and bandwidth. Each thread has access to private registers, while threads within a block can communicate through shared memory. All threads may access global memory, which provides large capacity but significantly higher latency than on-chip memory spaces (Kirk and Hwu, 2013).

Efficient utilization of the CUDA memory hierarchy is essential for achieving high performance. As will be shown in Section 1.2, rule mining over knowledge graphs repeatedly evaluates multi-atom triple patterns that generate large intermediate result sets. The performance of such operations is strongly influenced by memory bandwidth and data locality. Consequently, understanding the GPU memory hierarchy is directly relevant to assessing whether these join-heavy workloads can benefit from acceleration.

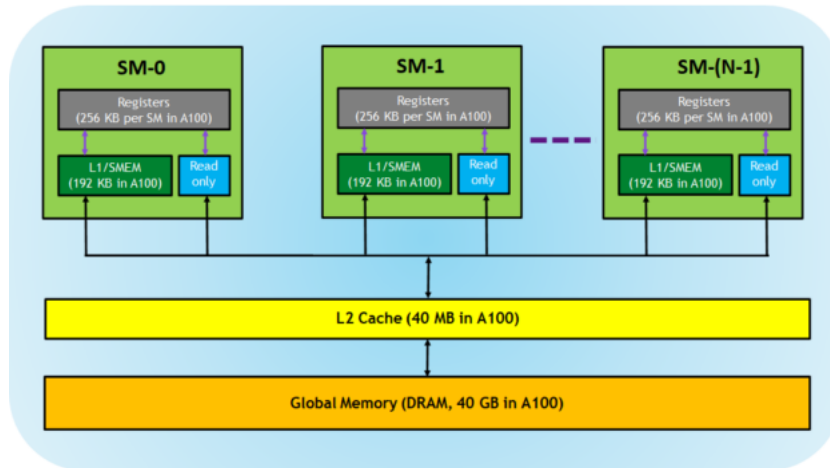
The CUDA programming model thus provides a practical foundation for exploiting GPU parallelism in general-purpose applications. The algorithms examined in this thesis contain computationally intensive components that operate on large data sets and exhibit a high degree of inherent parallelism. CUDA enables these components to be expressed as data-

parallel kernels and efficiently mapped onto GPU hardware, enabling significant acceleration compared to CPU-only execution.

Furthermore, the explicit control over thread organization, execution behavior, and memory usage offered by CUDA allows implementations to be tailored to the specific characteristics of the target algorithms. Concepts such as the thread hierarchy, SIMT execution, and memory organization directly influence the design decisions made in the practical part of this thesis, where selected stages of the rule mining process are accelerated while preserving correctness and scalability within a heterogeneous CPU–GPU environment.

1.1.3 GPU memory hierarchy

Modern GPUs employ a hierarchical memory system designed to support massively parallel execution while balancing memory access latency, bandwidth, and capacity. Unlike CPUs, which rely heavily on large cache hierarchies to reduce memory latency for a small number of threads, GPUs are designed to tolerate memory latency through massive concurrency. As a result, the GPU memory hierarchy provides multiple memory spaces with different performance characteristics and scopes, allowing developers to explicitly manage data placement and access patterns to achieve high throughput.



Schema 5 — Memory hierarchy on GPUs (NVIDIA Corporation, 2020)

At the lowest level of the hierarchy are registers, which are private to each thread and provide the fastest available storage. Registers are used to store local variables and intermediate results during kernel execution. Due to their limited size, excessive register usage can reduce the number of active threads and negatively affect occupancy. Above registers, CUDA provides shared memory, an on-chip memory space shared among threads

within the same block. Shared memory offers low-latency access and enables efficient data reuse and cooperation between threads, making it particularly useful for algorithms that repeatedly access the same data (Kirk and Hwu, 2013).

All threads executing on the GPU can access global memory, which resides in off-chip device memory and provides large storage capacity. However, global memory accesses incur significantly higher latency compared to on-chip memory spaces. To mitigate this cost, modern GPUs incorporate caching mechanisms that automatically cache portions of global memory. Nevertheless, efficient GPU programs are typically structured to minimize global memory accesses and to maximize reuse of data loaded into registers or shared memory (Cheng et al., 2014).

In addition to global memory, CUDA defines several specialized memory spaces optimized for specific access patterns. Constant memory is a read-only memory space cached on-chip and optimized for uniform access across threads, while texture memory is designed for spatially localized access patterns and benefits from dedicated caching hardware. These memory spaces allow further optimization when application data access exhibits predictable or structured behavior, although their effective use depends strongly on the characteristics of the workload.

Efficient utilization of the GPU memory hierarchy is a key factor in achieving high performance. Because memory latency cannot be eliminated entirely, CUDA applications rely on a combination of concurrency and careful memory placement to hide latency and maintain high utilization of execution units. By organizing data access patterns to align with the memory hierarchy and the SIMT execution model, developers can significantly improve performance and scalability. These considerations play an important role in the practical part of this thesis, where memory access behavior directly influences the effectiveness of GPU acceleration.

1.1.4 Applicability of GPUs to Different Computational Workloads

Graphics processing units are particularly effective for workloads that expose a high degree of data parallelism, where the same operation is applied independently to large numbers of data elements. Such workloads can be decomposed into many lightweight threads that execute similar instruction streams with regular memory access patterns. Examples include dense linear algebra operations, stencil computations, image and signal processing, Monte Carlo simulations, and other numerically intensive algorithms. In these cases, the SIMT execution model and high memory bandwidth of GPUs allow thousands of threads to make progress concurrently, while latency is hidden through warp scheduling and massive hardware multithreading (Cheng et al., 2014).

In contrast, workloads characterized by irregular control flow, frequent branching, strong sequential dependencies, or highly unpredictable memory access patterns tend to perform

less efficiently on GPUs. Warp divergence, which occurs when threads within a warp follow different execution paths, reduces execution efficiency because instructions must be serialized across divergent paths (Cheng et al., 2014). Similarly, applications dominated by pointer chasing, small problem sizes, or frequent synchronization may fail to provide sufficient parallel work to amortize kernel launch and data transfer overheads. Such workloads are often better suited for CPUs, which are optimized for low-latency execution, sophisticated branch prediction, and complex control flow.

The practical effectiveness of GPU acceleration therefore depends on several key factors: the amount of inherent parallelism available, the computational intensity relative to memory access cost, the regularity of control flow, and the ability to structure memory accesses in a coalesced manner. When these conditions are satisfied, GPUs can deliver substantial throughput improvements. When they are not, architectural characteristics such as SIMT execution and memory hierarchy constraints may limit achievable performance gains.

1.2 Graph Data and Rule Mining Background

1.2.1 Knowledge graphs and RDF-style data

Knowledge graphs represent structured information as collections of entities connected by labeled relationships. In most practical systems, such graphs are modeled using the Resource Description Framework (RDF), where data is expressed as triples of the form *(subject, predicate, object)*. Each triple corresponds to a binary relation between two entities, and the complete set of triples forms a directed, labeled, multi-relational graph. This representation allows knowledge graphs to encode heterogeneous information in a uniform relational format while preserving logical interpretability.

From a formal perspective, RDF data can be viewed as a set of ground atoms over binary predicates, making it suitable for logical reasoning and rule mining. In this setting, entities correspond to constants and predicates to relations in first-order logic. Rule mining systems such as AMIE exploit this structure by searching for Horn clauses that describe regularities in the data. A Horn rule typically has the form:

$$r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$$

where the body represents a graph pattern and the head represents a predicted relation. This logical interpretation enables the discovery of implicit knowledge from explicitly stored triples (Galárraga et al., 2015).

Large real-world knowledge graphs, such as DBpedia or Wikidata, contain millions or even billions of triples. Their structure is typically sparse, highly heterogeneous, and characterized by skewed degree distributions. These structural properties significantly affect the computational behavior of rule mining algorithms. In particular, the need to evaluate candidate rules requires repeated join operations over large triple sets, making performance heavily dependent on the efficiency of graph traversal and pattern matching (Galárraga et al., 2015).

Recent advances in rule mining frameworks have focused on improving both usability and computational efficiency. For example, RDFRules proposes techniques to simplify rule specification and optimize rule evaluation over RDF graphs, highlighting the importance of efficient support counting and candidate generation in large knowledge bases (Zeman et al., 2021). These systems emphasize that practical rule mining is not merely a logical problem but also a computational challenge shaped by the scale and structure of RDF data.

The relational and graph-based nature of RDF knowledge graphs therefore provides both the foundation for inductive rule learning and the source of its computational complexity. The structural characteristics of RDF knowledge graphs—particularly their scale, sparsity, and skewed degree distributions—directly influence the computational behavior of rule mining algorithms discussed in the following section. In particular, multi-way joins over large triple sets become the dominant cost factor when evaluating candidate rules.

1.2.2 Rule mining and Inductive Logic Programming Basics

Rule mining over knowledge graphs aims to automatically discover logical regularities that capture dependencies between relations in structured data. In RDF-based knowledge graphs, these regularities are typically expressed as Horn clauses over binary predicates. A Horn rule consists of a conjunction of relational atoms in its body implying a single relational atom in its head. For example, a rule of the form

$$r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$$

states that whenever two entities are connected through relations r_1 and r_2 , they are likely also connected through relation r_3 . Because RDF data is naturally represented as triples, each atom in such a rule corresponds directly to a triple pattern, making rule mining equivalent to discovering frequently co-occurring patterns of joins in the graph (Galárraga et al., 2015).

From a methodological perspective, rule mining in knowledge graphs is closely related to Inductive Logic Programming (ILP). ILP seeks to induce logical rules from observed data by generalizing from examples. However, unlike classical ILP settings that rely on clearly defined positive and negative examples, large-scale knowledge graphs are inherently

incomplete. The absence of a fact in an RDF graph does not necessarily imply that the fact is false. This incompleteness has significant implications for rule evaluation and distinguishes knowledge graph rule mining from traditional supervised learning approaches (Galárraga et al., 2015).

Modern rule mining systems, such as AMIE and its successor AMIE+, address scalability and incompleteness challenges by restricting the hypothesis space to function-free Horn clauses and by defining efficient refinement operators that incrementally construct candidate rules. The search process typically begins with simple rules containing a single atom in the body and progressively extends them by adding new atoms. Each extension corresponds to introducing additional join constraints over the graph. This refinement-based search enables systematic exploration of the rule space while maintaining logical consistency and connectedness between variables (Galárraga et al., 2015).

To evaluate candidate rules, statistical measures such as support and confidence are employed. Support reflects how many instances in the graph satisfy both the body and the head of the rule, which in practice corresponds to evaluating multi-way joins over RDF triples. Confidence measures the reliability of the rule, that is, the proportion of cases in which the body implies the head. However, due to the open-world nature of knowledge graphs, standard confidence measures may underestimate rule quality. To address this issue, AMIE introduced the Partial Completeness Assumption (PCA) confidence, which adjusts the evaluation to account for known facts rather than assuming closed-world completeness (Galárraga et al., 2015). This adaptation has become fundamental in large-scale rule mining over incomplete knowledge bases.

Beyond individual systems, the broader literature emphasizes that rule mining over knowledge graphs must balance expressiveness and computational feasibility. The search space of possible Horn clauses grows combinatorially with the number of relations and variables, and evaluating each candidate rule requires executing join operations over large triple stores. Therefore, effective pruning strategies and efficient query evaluation mechanisms are essential for scalability (Zeman et al., 2021). In practice, the computational complexity of rule mining is dominated not by logical reasoning itself, but by repeated pattern matching and counting operations over the underlying graph data.

In summary, rule mining in RDF-style knowledge graphs can be understood as a structured search over conjunctive relational patterns, guided by statistical evaluation metrics and constrained by scalability considerations. The combination of logical expressiveness, incomplete data assumptions, and large-scale join operations makes rule mining both conceptually grounded in ILP and computationally demanding. These characteristics are particularly relevant for this thesis, as the repeated evaluation of multi-atom rule bodies constitutes a major computational bottleneck and therefore represents a promising target for GPU acceleration.

1.2.3 Computational bottlenecks on CPU

Although rule mining over knowledge graphs is formally grounded in logical inference, its practical performance is determined primarily by large-scale data processing. In systems such as AMIE and AMIE+, rule discovery proceeds as an iterative exploration of a large hypothesis space, where candidate rules are constructed and evaluated against the underlying RDF graph (Galárraga et al., 2015). The computational effort is therefore concentrated not in symbolic reasoning itself, but in repeatedly evaluating conjunctive patterns over large triple collections.

A central mechanism for controlling the size of this search space is the minimum support threshold. A candidate rule is retained and extended only if its support — the number of distinct head-pair groundings satisfying the full rule body — meets the configured minimum. This threshold directly determines the trade-off between the quantity and quality of the output rule set: a lower threshold yields more rules and potentially better predictive coverage, but at substantially higher computational cost, because far more candidates must be evaluated and extended. Choosing an appropriate threshold is therefore critical to making the mining process tractable. RDRules addresses this challenge through a top-k approach combined with rule patterns, which allow the user to guide the search toward the most relevant regions of the rule space without enumerating all candidates exhaustively (Zeman et al., 2021).

An important asymmetry exists between the two primary interestingness measures. Support is anti-monotonic: if a rule body fails to meet the minimum support threshold, no extension of that body can meet it either, so the entire subtree can be safely pruned. Confidence does not share this property — a short rule with low confidence may have longer extensions with high confidence, so the state space cannot be efficiently pruned on the basis of a minimum confidence threshold alone. This means the mining process must generate a large number of candidate rules and compute support and confidence for each, retaining only those that meet the desired thresholds in post-processing. The practical consequence is that support computation and body-size enumeration, which together determine confidence, must be performed for a very large number of candidates, many of which will ultimately be discarded.

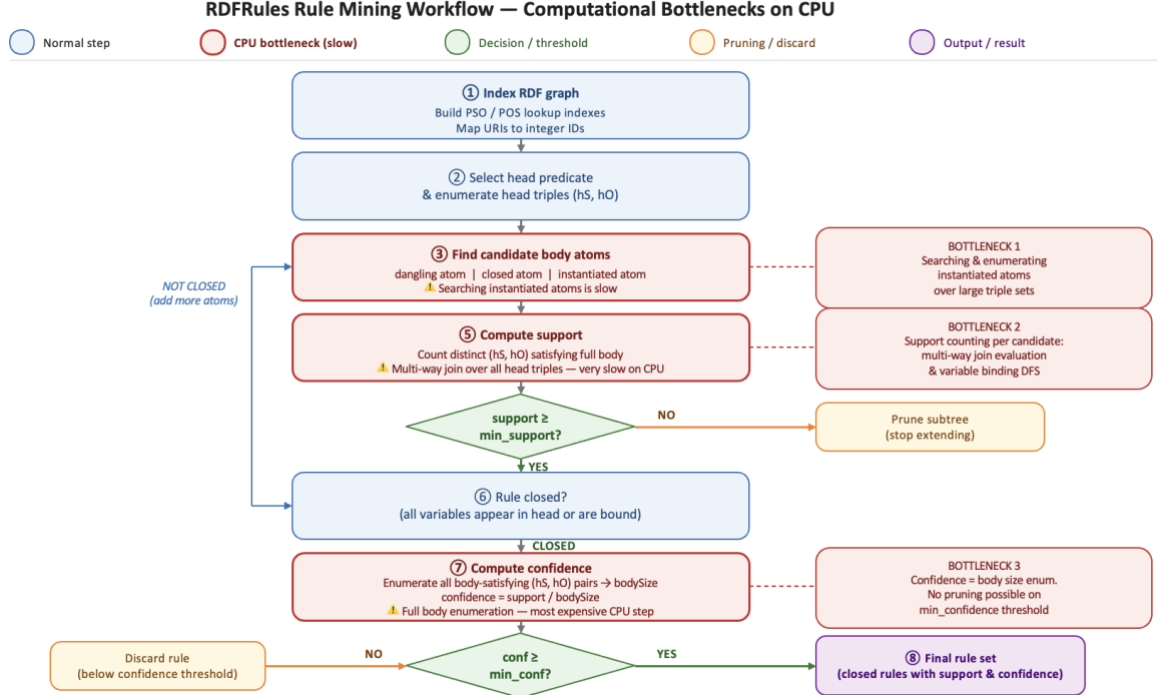
The evaluation of rule bodies corresponds to executing multi-way joins over RDF triples. Each additional atom in the body introduces further join constraints that must be satisfied simultaneously, requiring scanning indexed triple tables, matching variable bindings, and constructing intermediate result sets. As rule bodies grow longer, the number of intermediate bindings may increase substantially before pruning takes effect, leading to high memory traffic and significant pressure on CPU cache hierarchies, particularly when intermediate results exceed cache capacity and must be stored in main memory.

The combinatorial nature of the search process further amplifies the computational cost. The number of possible rule candidates grows rapidly with the number of relations in the knowledge graph and with the allowed body length. While minimum support pruning eliminates subtrees that cannot reach the threshold, it does not reduce the cost of evaluating candidates that do pass support checking. Because confidence cannot be used for early pruning, a large number of rule bodies must be fully evaluated — their support counted and their body size enumerated — before any confidence-based filtering can take place (Galárraga et al., 2015). The bottleneck is therefore not rule generation logic itself, but the repeated expansion and evaluation of candidate rules, particularly when longer, multi-atom rules are being searched.

Another important factor is the irregular structure of real-world knowledge graphs. Degree distributions are often highly skewed and relation frequencies vary considerably, so different candidate rules can produce vastly different intermediate result sizes. Some patterns yield only a handful of bindings, while others generate large intermediate joins, producing unpredictable memory access patterns and fluctuating workload sizes. Although CPUs are optimized for complex control flow and low-latency branching, performance degrades when workloads become dominated by large, memory-bound scans rather than compute-heavy operations. When the working set exceeds cache capacity, execution speed is constrained by memory bandwidth and latency rather than arithmetic throughput, and even well-designed indexing strategies cannot fully eliminate this bottleneck.

In addition to join processing, rule mining systems must compute statistical measures such as support and confidence for each candidate rule, involving counting and aggregation over sets of bindings that may themselves be large. Because these statistics must be evaluated for many candidates and confidence cannot prune the search space, the cumulative cost becomes substantial. In large-scale knowledge bases, a significant fraction of total execution time is spent in support and confidence computation rather than in rule generation

logic — and this is precisely the phase targeted by the implementations developed in this thesis.



Schema 6 — RDFRules rule mining workflow: computational bottlenecks on CPU (own processing)

Taken together, rule mining on CPUs can be characterized as a workload dominated by repeated join evaluation, extensive memory access, and combinatorial search. The search space can be pruned using the minimum support threshold, but confidence-based pruning is not available, so a large number of candidate rules must be fully evaluated regardless. While CPUs are well suited for managing complex search logic and dynamic refinement decisions, the data-parallel nature of join evaluation and counting suggests that other architectures may be able to process these phases more efficiently. These operations exhibit substantial data-level parallelism — bindings can be processed independently and aggregation distributed across data partitions — and they align closely with the architectural strengths of GPUs described in Section 1.1, particularly their high memory bandwidth and ability to execute thousands of concurrent threads. This observation motivates the investigation of GPU-based acceleration strategies in the practical part of this thesis.

1.3 Key Software Libraries and Runtime Dependencies

The implementations developed in this thesis rely on a set of well-established software libraries and runtime components provided by NVIDIA and the broader high-performance

computing ecosystem. Understanding these libraries is important for situating the implementation decisions made in the practical chapters and for assessing the portability and reproducibility of the results.

1.3.1 NVIDIA CUDA Toolkit and NVCC

All GPU implementations in this thesis are compiled using NVCC, the NVIDIA CUDA compiler driver. NVCC separates device code from host code at compile time and invokes the appropriate backend compiler for each. Device-side code is compiled to PTX (Parallel Thread Execution), NVIDIA’s intermediate representation, which is subsequently assembled into GPU machine code for the target architecture. Host-side code is forwarded to a standard C++ compiler such as GCC. This heterogeneous compilation model allows CUDA programs to mix device kernels and conventional C++ logic within the same source files.

The CUDA runtime library, which is part of the toolkit, provides the application programming interface used throughout the GPU implementations: CUDA memory allocation and transfer functions (`cudaMalloc`, `cudaMemcpy`), kernel launch syntax, event-based timing via `cudaEvent_t`, and atomic operations. All GPU implementations in this thesis target CUDA compute capabilities supported by the NVIDIA Hopper architecture (compute capability 9.0, as used by the H200 NVL GPU) and are compiled with the `-O3` optimization flag and the C++17 standard.

1.3.2 Thrust

Thrust is a parallel algorithms library included with the CUDA Toolkit that provides GPU-accelerated equivalents of C++ Standard Template Library algorithms such as `sort`, `reduce`, and `unique`. Thrust abstracts away the low-level kernel launch and thread management details required for these common operations, allowing them to be invoked with a single function call while still executing on the GPU. In this thesis, Thrust is used in the fallback body-size counting path of the GPU implementation, where pairs of entity identifiers must be deduplicated after a kernel collects all body groundings. The combination of `thrust::sort` and `thrust::unique` performs this deduplication efficiently on the GPU without requiring a data round-trip to CPU memory.

1.3.3 cuSPARSE

cuSPARSE is NVIDIA’s library for sparse linear algebra operations on the GPU. It provides optimised implementations of operations such as sparse matrix-vector multiplication and sparse matrix-matrix multiplication (SpGEMM) for matrices stored in standard formats such as Compressed Sparse Row (CSR). In this thesis, cuSPARSE was investigated as the basis for an algebraically motivated approach to rule support counting,

in which each predicate in a knowledge graph is represented as a sparse adjacency matrix and rule body evaluation is reformulated as a sequence of matrix products. As discussed in the implementation chapters, this approach proved infeasible at the scale of the benchmark dataset due to intermediate matrix explosion. Nevertheless, the exploration of cuSPARSE informed the design of the final GPU implementation by establishing the suitability of the CSR data format for predicate indexing on the GPU.

1.3.4 Java Native Access

Java Native Access (JNA) is a standard Java library that enables JVM programs to call functions in native shared libraries compiled from C or C++ without requiring the developer to write Java Native Interface (JNI) boilerplate code. JNA uses reflection to map Java interface declarations to native function signatures at runtime, handling type conversion between Java and C types automatically. In this thesis, JNA serves as the integration bridge between the Scala-based RDRules framework, which runs on the JVM, and the native C++/CUDA support counting library. The integration design and its practical usage are described in detail in the dedicated integration chapter.

1.4 Related work

The problem of accelerating rule mining over large knowledge graphs has attracted considerable attention, and several distinct strategies have been explored in the literature.

The most directly related line of work consists of systems that preserve the exhaustive, exact character of AMIE-style rule mining while improving computational efficiency. AMIE 3 (Lajus and Galárraga, 2020) introduces pruning strategies, multi-threaded parallelisation across candidate rules, and a lazy evaluation scheme that defers confidence computation until needed, reducing runtime by more than fifteen times compared to AMIE+ on benchmarks such as DBpedia and Wikidata while still computing exact support and confidence values for every rule. The Ontological Pathfinding algorithm (Chen et al., 2016) takes a different approach: it represents the knowledge base in a relational model, parallelises join queries across sub-tasks using a graph partitioning scheme, and applies upfront pruning to eliminate logically unsound rules before evaluation — making it the first system to mine inference rules from Freebase at the scale of 388 million facts. AnyBURL (Meilicke et al., 2019) addresses scalability from yet another angle: rather than exhaustively exploring the rule space top-down, it samples random paths through the knowledge graph and generalises them into Horn rules bottom-up, yielding an anytime algorithm that produces increasingly complete rule sets over time and remains competitive with embedding-based approaches in link prediction quality while staying fully symbolic and interpretable.

A separate and large body of related work approaches knowledge graph reasoning through graph embeddings. Systems such as TransE (Bordes et al., 2013) and RotatE (Sun et al., 2019) represent entities and relations as vectors or rotations in a continuous low-dimensional space and use GPU-accelerated matrix operations and gradient-based optimisation to train scoring functions that assign plausibility to candidate triples. These methods are highly scalable and have produced strong results on link prediction benchmarks. However, a fundamental distinction separates them from the exhaustive approach of AMIE and RDFSRules. Because the entire knowledge graph is compressed into a low-dimensional embedding space, any relational patterns or rule-like structures that can be extracted from these representations are approximations shaped by the choice of dimensionality, the training objective, and the scoring function rather than by the actual distribution of triples in the graph. Support and confidence values derived from embeddings are therefore statistical estimates, not exact counts. AMIE and RDFSRules, by contrast, compute exact support and confidence over the original triple store without any compression or intermediate representation, ensuring that the resulting rules and their interestingness measures are not distorted. This exactness matters whenever mined rules are intended for downstream reasoning, knowledge base completion, or explanation generation, where biased confidence values could lead to incorrect inferences. The present thesis operates entirely within this exhaustive paradigm and focuses on reducing the computational cost of exact support and confidence computation rather than replacing it with an approximation.

2 Implementation and Optimization of Support Counting

This chapter presents the practical contribution of the thesis: a sequence of implementations of support and confidence counting for inductive logic rule mining over RDF graphs. The work starts from the existing Scala RDRules framework and then develops three native implementations: a faithful C++ rewrite, a cache-optimized non-parallel CPU version, and the final CUDA GPU implementation. The intention is not to replace the whole RDRules mining pipeline, but to accelerate the most expensive counting step while preserving the same rule semantics.

The source of truth for all implementations is the original RDRules logic. A rule is treated as a Horn clause whose body and head consist of RDF-style binary atoms. Support is the number of distinct head subject-object pairs for which the head triple exists and the body has at least one valid grounding. Body size is the number of distinct head-variable pairs satisfying the body independently of whether the head triple exists. Confidence is computed as support divided by body size. All implementations therefore have to agree on support, body size, and confidence, not only on runtime.

The practical work follows an iterative optimization strategy. First, the Scala implementation is used as a correctness reference. Second, the same algorithm is rewritten in C++ in order to isolate the cost of the JVM runtime. Third, the CPU version is redesigned around cache-friendly data structures. Finally, the same counting problem is mapped to the GPU using CUDA kernels and graph indexes stored in device memory.

2.1 Scala Baseline: RDRules Library

2.1.1 Role of RDRules in the Thesis

RDRules is the original Scala/JVM rule mining framework used as the baseline in this thesis. It performs rule generation, candidate filtering, and metric computation over RDF knowledge graphs. This thesis does not reimplement the full mining pipeline. Instead, it focuses on replacing the support and confidence counting phase with native CPU and GPU implementations while leaving rule generation and the surrounding RDRules workflow conceptually unchanged.

The Scala implementation is important for two reasons. First, it defines the expected semantics of rule evaluation. Second, it provides the initial performance reference against which the C++ and CUDA implementations are compared. The later implementations are

therefore considered correct only if they produce the same metric values for the same input graph and rules.

2.1.2 Data Loading and Index Construction

RDFRules loads RDF data from files such as Turtle or N-Triples, maps URI strings and literals to compact internal identifiers, and builds indexes used during rule evaluation. Conceptually, the relevant indexes are predicate-oriented maps in two directions: subject-to-objects and object-to-subjects. These correspond to PSO and POS access patterns. For a fixed predicate, the PSO index allows lookup of all objects connected to a given subject, while the POS index allows lookup of all subjects connected to a given object.

The JVM implementation stores these structures through nested hash maps and collection objects. This design is flexible and convenient, but it creates many heap objects and pointer indirections. In small workloads this overhead is acceptable, but in rule counting it becomes significant because the DFS traversal performs a very large number of index lookups.

2.1.3 Rule Representation and Counting Semantics

Rules are represented as body atoms implying one head atom. Each atom consists of a predicate and two arguments, where each argument may be a variable or a constant. During evaluation, variables are bound to integer entity identifiers. The implementation enforces injective mapping, meaning that two different variables cannot be bound to the same entity, and the same ground triple cannot be reused incorrectly during one grounding.

Support counting begins by iterating over all triples of the head predicate. For each head triple, the head variables are bound to the subject and object of that triple. The algorithm then runs a depth-first search over the body atoms and checks whether at least one complete grounding exists. If a valid grounding is found, the search stops for that head triple and support is incremented. This early-exit behavior makes support counting substantially cheaper than confidence counting.

Body-size counting is more expensive. It evaluates the body without requiring the head triple to exist and records all distinct pairs corresponding to the head variables. These pairs are deduplicated and counted. Since the algorithm cannot stop after the first grounding, confidence computation often dominates total runtime.

2.1.4 Performance Characteristics

The Scala implementation is functionally correct but not ideal for the counting workload. JVM object allocation, garbage collection, boxed values, nested hash maps, and indirect memory accesses all increase overhead. The rule counting computation is dominated by

repeated graph lookups and recursive DFS traversal, so memory locality is crucial. These limitations motivate the native C++ rewrite and the later memory-layout optimizations.

2.2 Faithful C++ Rewrite

2.2.1 Design Goal

The first native implementation is the Rewritten C++ version. Its purpose is not to introduce a new algorithm, but to reproduce the original Scala/RDFRules counting logic as faithfully as possible. The graph is stored in two nested hash-map indexes, PSO and POS, and the same DFS approach is used for both support and body-size counting.

This implementation isolates the effect of moving from Scala/JVM to native C++. If the algorithm and data structures remain nearly identical, any speedup can mainly be attributed to removing JVM overhead, reducing object allocation costs, and using native integer containers instead of JVM collections.

2.2.2 Graph Storage

The RDF graph is loaded into integer identifiers using an ID mapper. The two main indexes are:

- PSO index: predicate -> subject -> list of objects.
- POS index: predicate -> object -> list of subjects.

These structures mirror the original RDFRules organization. However, they are still pointer-heavy and cache-unfriendly. A single lookup may require traversing the outer predicate map, then an inner subject or object map, and finally a vector containing candidate entities. This preserves correctness but does not solve the memory-locality bottleneck.

2.2.3 Support and Body-Size Counting

Support counting iterates over triples of the head predicate. For each head pair, the corresponding head variables are bound, and the recursive DFS attempts to satisfy all body atoms. The DFS chooses an unsatisfied atom, looks up candidate triples through PSO or POS, applies variable bindings, checks injective mapping, and backtracks when necessary. If any full grounding is found, the current head pair contributes one to support.

Body-size counting uses the same DFS traversal but starts without binding the head variables from an existing head triple. At each complete body grounding, the pair corresponding to the head variables is inserted into a hash set. The final body size is the

number of distinct pairs in this set. Confidence is then computed as support divided by body size.

2.2.4 Correctness Role

The faithful C++ rewrite is used as the main native correctness baseline. The optimized CPU implementation and the GPU implementation are expected to produce the same support, body-size, and confidence values as this version. This is important because the later implementations change memory layout and execution strategy significantly.

2.2.5 Limitations

Although the C++ rewrite removes much of the JVM overhead, it keeps the main structural bottlenecks. The nested `unordered_map` layout leads to poor cache locality. The recursive DFS uses dynamic data structures for bindings and state. Atom tracking and binding management are generic rather than specialized for the small rule body sizes typical in the benchmark sets. These limitations motivate the optimized CPU implementation.

2.3 Optimized Non-Parallel CPU Implementation

The optimized CPU implementation, referred to as `CPU_01_nonparallel`, keeps the same rule evaluation semantics but redesigns the data structures and hot DFS state. It is still single-threaded. The goal is to determine how much speedup can be achieved on the CPU before using GPU parallelism.

2.3.1 CSR Graph Layout

The largest change is replacing nested hash maps with a Compressed Sparse Row style layout. For each predicate and direction, the index is represented by three contiguous arrays:

- keys: sorted unique subjects or objects.
- offsets: positions where each key group begins in the value array.
- values: all matching objects or subjects stored contiguously.

For example, the PSO direction stores all objects grouped by subject. A lookup first performs a binary search over the sorted keys array and then returns a contiguous slice from the values array. Although this changes lookup from hash-based amortized constant time to logarithmic search over keys, the data is much more cache-friendly because the arrays are contiguous.

This layout is particularly effective for dense predicates and long adjacency lists. However, it is not universally better. As shown in the experimental evaluation, CSR can regress on high-fanout predicates when the traversal enumerates many intermediate candidates that the hash-map DFS would prune earlier.

2.3.2 Stack-Local DFS Environment

The optimized CPU implementation replaces heap-allocated binding structures with a fixed-size stack-local environment. Variable bindings are stored in a small integer array, with zero representing an unbound variable and bound entity identifiers stored with an offset. This avoids ambiguity with entity identifier zero and makes binding checks cheap.

During DFS backtracking, the implementation saves previous binding values and restores them after the recursive call returns. This avoids copying heap-allocated vectors at every recursion level. Since rules in the experiments have small body sizes, the full DFS state fits comfortably in cache.

2.3.3 Bitmask Atom Tracking

The implementation tracks which body atoms remain unmatched using a compact bitmask instead of a vector of booleans. Each bit corresponds to one body atom. Checking, clearing, and restoring atom state are therefore register-level operations. Because the bitmask is passed by value during recursion, backtracking is natural and does not require explicit reconstruction of a used-atom array.

2.3.4 Template-Specialized DFS

The DFS functions are specialized by rule body size using C++ templates. Separate versions are generated for small body sizes such as one, two, and three atoms. This allows the compiler to optimize recursion depth, loop bounds, and branch structure more aggressively than in one fully generic function. The implementation therefore reduces overhead in the inner loop without changing the search semantics.

2.3.5 Expected Advantages and Risks

The optimized CPU implementation improves memory locality, reduces heap allocation, and simplifies state tracking. On dense synthetic graphs this produces a clear speedup over the faithful C++ rewrite. However, the optimization is topology-dependent. CSR traversal may visit large contiguous candidate ranges even when an alternative traversal order or hash-map behavior prunes earlier. For this reason, the optimized CPU implementation is not automatically superior on all datasets.

Optimized CPU Implementation — Data Structures

① CSR Graph Index

rowPtr	0	3	5	8	10	13			contiguous arrays O(1) neighbour access no pointer chasing
colIdx	1	2	4	0	3	1	4	5	
labels	A	B	A	C	B	A			

② Stack-Local Env Struct

mapping[N]	atomMask	depth	queryNbrs	hostNbrs
------------	----------	-------	-----------	----------

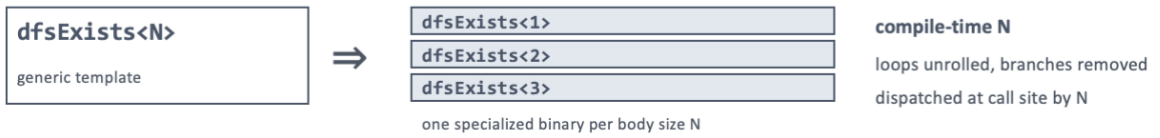
one struct per DFS call — lives on the stack

③ Bitmask Atom Tracking

atomMask	0	1	1	0	1	0	0	1	0	0	1	0	1	1	0	0	...
----------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

64-bit register — 1 bit per matched host atom

④ Template-Specialized DFS



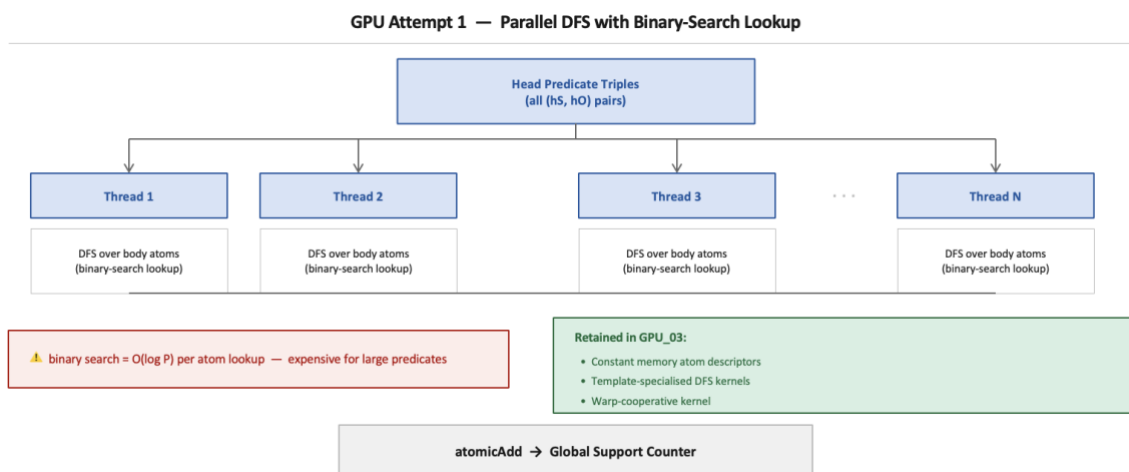
Schema 7 — CPU implementation (own processing)

2.4 GPU Implementation Trajectory

Before the final CUDA implementation was reached, two earlier GPU approaches were tested. These unsuccessful or partially successful experiments are important because they explain why the final GPU design has its current structure. They also show that GPU acceleration is not obtained simply by moving existing code to CUDA; the data layout and execution model must match the graph workload.

2.4.1 GPU Attempt 1: Parallel DFS with Binary-Search Lookup

The first GPU implementation established the main parallelization idea. Support counting can be parallelized over head triples because each head triple can be tested independently. A CUDA kernel can therefore assign one thread, or in high-fanout cases one warp, to a head-predicate pair. Each thread binds the head variables and runs a compact DFS over the body atoms.



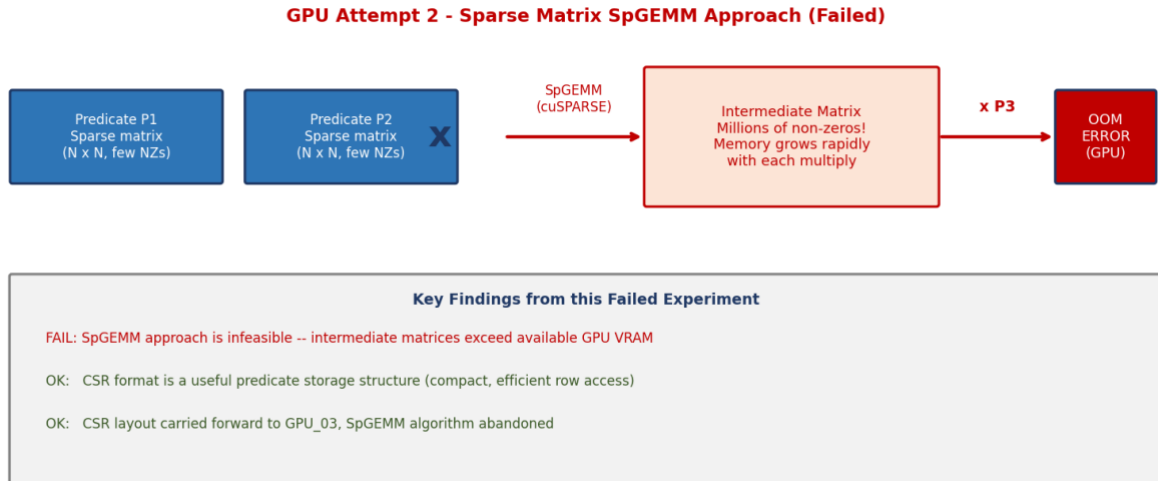
Schema 8 — GPU Attempt 1: parallel DFS with binary-search lookup (own processing)

This version introduced several ideas that were retained in the final implementation: rule atom descriptors in CUDA constant memory, register-resident variable bindings, template-specialized DFS kernels, warp-level reduction, and a warp-cooperative mode for predicates with high fanout. These ideas map well to the GPU because many head pairs can be processed simultaneously.

The limitation was the predicate storage format. Predicate triples were stored as sorted subject-object pairs. To find the range of objects for a subject, the GPU had to perform binary searches over the pair array. For large predicates this added many memory reads and comparisons inside the DFS. The GPU was parallel, but each thread still paid unnecessary lookup cost.

2.4.2 GPU Attempt 2: Sparse Matrix Formulation

The second GPU experiment used a sparse-matrix interpretation of rule evaluation. Each predicate was represented as a sparse adjacency matrix, and chain-like rule bodies were evaluated through sparse matrix multiplication. This approach was attractive because sparse matrix multiplication is a well-known GPU workload supported by libraries such as cuSPARSE.



Schema 9 — GPU Attempt 2: SpGEMM approach and CSR discovery (own processing)

In practice, the approach failed for two interconnected reasons. The first was intermediate matrix explosion. Even if the original predicate matrices are sparse, the product of two sparse adjacency matrices over a knowledge graph may become much denser. For predicates with many-to-many relationships, intermediate products can contain millions or more non-zero entries. On the benchmark graphs, the GPU memory requirements became too large, and the computation failed before producing useful results.

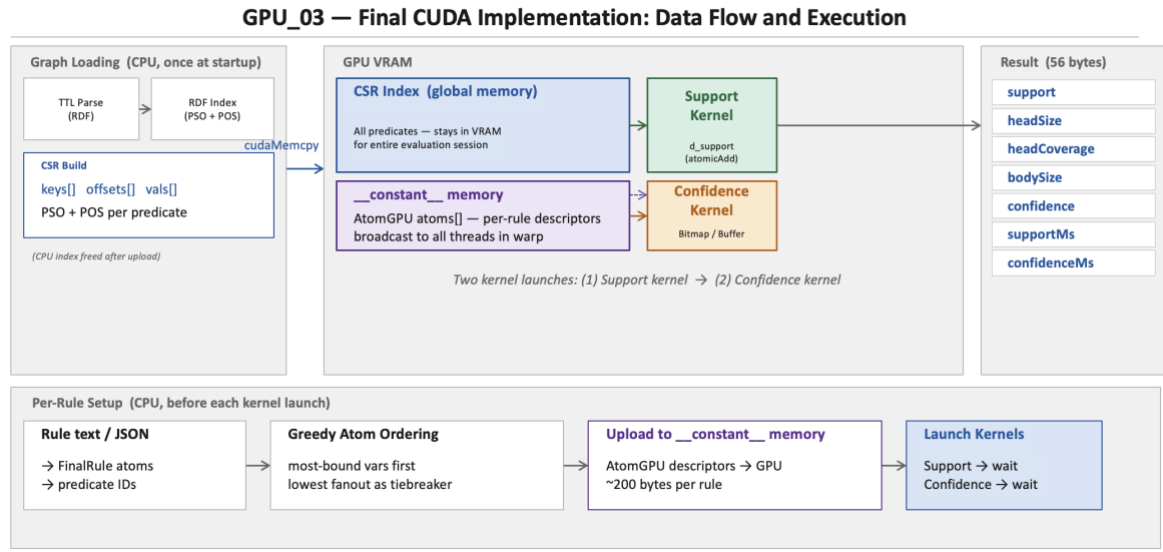
The second problem was injective mapping. The DFS-based algorithm enforces that different variables in a rule body must bind to different entities, the same entity cannot fill two distinct variable slots simultaneously. In the matrix multiplication formulation, no equivalent mechanism exists. The algebraic operations produce counts of path-joining solutions without tracking which entities appear at which positions, making it impossible to filter out non-injective bindings without first materializing all intermediate results and then post-processing them. This would have eliminated the main efficiency advantage of the approach.

Together, these two issues made the SpGEMM formulation fundamentally unsuitable for the target workload. The experiment was nonetheless useful. It confirmed that CSR is an effective graph storage format because it provides compact representation and efficient $O(1)$ row access. The final GPU implementation therefore retained CSR indexing but abandoned sparse matrix multiplication entirely, returning to the DFS-based evaluation strategy with GPU parallelism applied at the level of individual head triples.

2.5 Final GPU Implementation: GPU_03

2.5.1 Design Rationale

GPU_03 is the final CUDA implementation used in the evaluation. It combines the useful parts of the earlier GPU attempts: the parallel DFS execution strategy from the first GPU version and the CSR graph representation discovered during the sparse-matrix experiment. The graph is loaded once, converted to CSR indexes, uploaded to GPU memory, and kept there for the lifetime of the evaluation context.



Schema 10 — GPU_03 final design: data flow and execution (own processing)

For each predicate, both directions are stored: PSO for subject-to-objects lookup and POS for object-to-subjects lookup. Each direction contains keys, offsets, and values arrays. The GPU kernels use these arrays to enumerate candidates during DFS. Small rule-specific descriptors are copied into CUDA constant memory before each kernel launch so that all threads can read the same atom metadata efficiently.

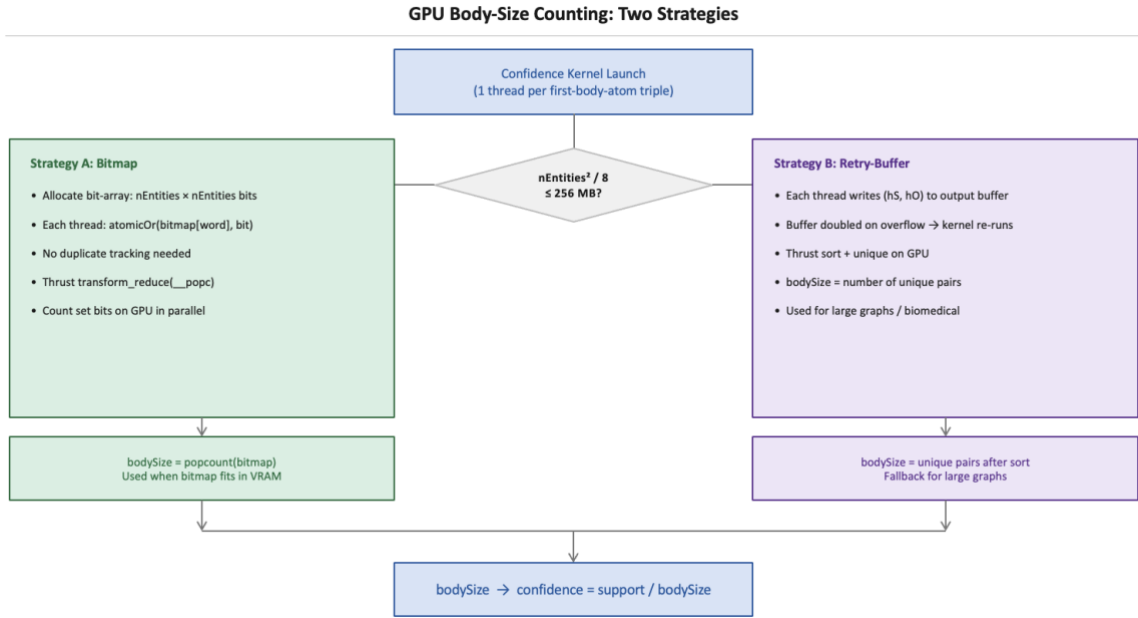
2.5.2 Support Counting on GPU

Support counting uses one of two kernel variants. The standard kernel assigns one CUDA thread to one head-predicate triple. The thread initializes bindings from the head subject and object, evaluates the body using DFS over CSR indexes, and contributes to a global support counter if the body is satisfied. Warp-level reduction is used to reduce contention before atomic updates.

For dense predicates with high fanout, GPU_03 uses a warp-cooperative kernel. In this mode, one warp is assigned to one head triple. The lanes cooperate to explore candidates, and the warp can terminate early using CUDA warp-vote instructions once any lane finds a valid grounding. This is useful when a single thread would otherwise spend too long traversing a large adjacency list.

2.5.3 Body-Size and Confidence Counting on GPU

Body-size counting is more difficult than support counting because all distinct body-induced head pairs must be counted. GPU_03 uses two strategies depending on graph size and available memory.

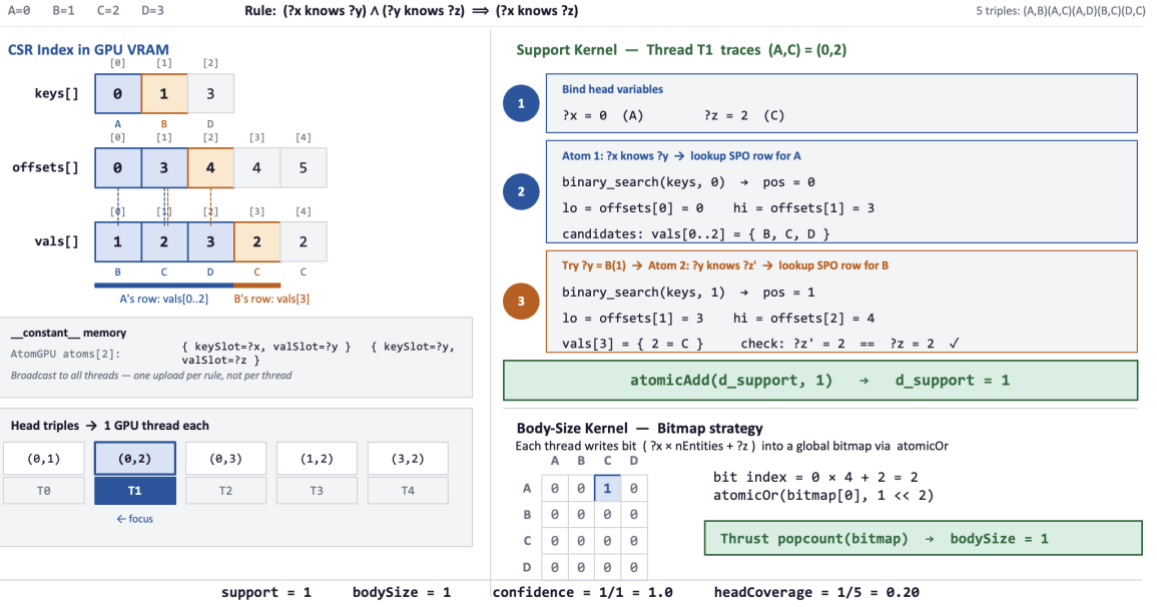


Schema 11 — Body-size counting: bitmap and retry-buffer strategies (own processing)

The bitmap strategy is used when the possible pair space can fit into GPU memory. Each discovered pair sets one bit in a global bitmap using atomic operations. After the kernel completes, Thrust is used to count set bits in parallel. This avoids storing duplicates explicitly and is very efficient for graphs where the bitmap is feasible.

For larger graphs, GPU_03 uses a retry-buffer strategy. Threads write discovered pairs into a large device buffer. If the buffer overflows, it is reallocated with a larger capacity and the kernel is repeated. After collection, Thrust sort and unique operations deduplicate the pairs on the GPU. This strategy is more memory-intensive but makes large graphs feasible without requiring an impossible full bitmap.

GPU_03 — CSR-Indexed DFS: How Support & Confidence Counting Works



Schema 12 — GPU_03 CSR-indexed DFS: step-by-step support and confidence counting on a small example (own processing)

2.5.4 CUDA-Specific Techniques

The final GPU implementation uses several CUDA-specific techniques to make the DFS-based counting algorithm efficient on the GPU. Rule-specific atom descriptors are stored in CUDA constant memory, because all threads evaluating the same rule need to access the same small metadata. This allows efficient broadcast within a warp and reduces unnecessary global memory reads.

The graph indexes are stored in GPU global memory in CSR format and remain there for the lifetime of the evaluation context. This avoids repeated host-device transfers for every rule. Only small rule-specific data has to be copied before each kernel launch. The main drawback is that the complete CSR index must fit into available GPU memory.

During DFS evaluation, variable bindings are kept in registers whenever possible. Since the evaluated rules contain only a small number of variables, this reduces global memory traffic in the innermost part of the search. Atomic operations are used to accumulate shared results, such as support counters and bitmap entries for body-size counting.

The implementation also uses Thrust for GPU-side aggregation. In the bitmap strategy, Thrust counts set bits using parallel reduction. In the retry-buffer strategy, Thrust sort and unique are used to deduplicate discovered pairs directly on the GPU. CUDA event timing is used to measure support and confidence kernel times separately from graph loading and CSR upload costs.

Together, these techniques keep most of the counting work on the GPU and reduce unnecessary memory movement between CPU and GPU.

2.5.5 Trade-Offs

The final GPU implementation is strongest when the workload exposes a large amount of independent work, especially when confidence computation generates many candidate body pairs. The one-time graph loading and CSR upload cost is significant, but it is amortized when many rules are evaluated. The main limitation is memory: the full CSR index must fit into GPU VRAM, and the retry-buffer strategy can require large temporary buffers for extreme rules.

The design therefore suits the intended use case: a rule mining pipeline where many candidate rules are evaluated against the same graph. It is less attractive for very small workloads where GPU setup dominates total runtime.

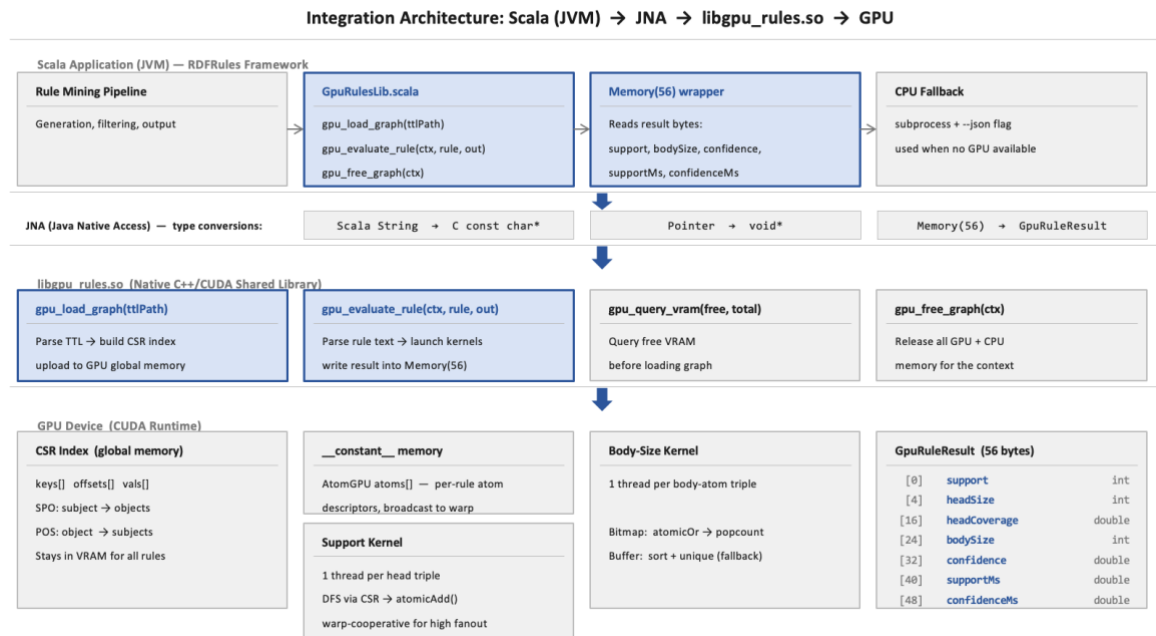
3 Integration with the Scala RDRules Framework

The optimized GPU implementation is written in C++ and CUDA, while RDRules is written in Scala and runs on the JVM. These two environments cannot call each other directly without an interoperability layer. The integration therefore compiles the CUDA implementation as a native shared library and loads it from Scala using Java Native Access (JNA).

3.1 Architecture Overview

The integration is designed as a drop-in replacement for the counting phase. RDRules remains responsible for graph mining logic, candidate rule generation, filtering, and output handling. The native backend receives a graph path and rule text or rule JSON, evaluates the metrics, and returns the results to Scala.

Schema 13 — Scala to native GPU backend integration (own processing):



Schema 13 — Full integration architecture: Scala JVM to GPU via JNA (own processing)

3.2 Compiling the Native Library

The CUDA code is compiled into a shared object file, `libgpu_rules.so`, using NVCC with the `-shared` and `-fPIC` flags. The resulting library exports a small C API through which the JVM can call into the GPU backend via JNA without any additional serialisation or inter-process communication. The API follows a load-once, evaluate-many pattern: the graph is parsed and uploaded to GPU memory once at startup, and individual rules are then evaluated against the resident index through successive calls. The five exported functions, their parameters, and their return values are listed in the table below.

Function	Parameters	Returns / Output
<code>gpu_load_graph</code>	<code>const char* ttlPath</code>	Opaque <code>void*</code> context pointer; NULL on failure
<code>gpu_evaluate_rule</code>	<code>void* ctx const char* ruleText GpuRuleResult* out</code>	Via <code>out</code> : <code>support</code> , <code>headSize</code> , <code>headSupport</code> , <code>bodySize</code> , <code>headCoverage</code> , <code>confidence</code> , <code>supportMs</code> , <code>confidenceMs</code>
<code>gpu_evaluate_rule_json</code>	<code>void* ctx const char* ruleJson GpuRuleResult* out</code>	Same fields as above; rule accepted as JSON string instead of text format
<code>gpu_query_vram</code>	<code>size_t* freeBytesOut size_t* totalBytesOut</code>	Via pointers: free and total VRAM in bytes; returns <code>-1</code> if no CUDA device found
<code>gpu_free_graph</code>	<code>void* ctx</code>	<code>void</code> — releases all host and device memory

Table 1 - C API functions exported by `libgpu_rules.so`.

3.3 Loading the Library from Scala

On the Scala side, JNA is added as a dependency. A Scala trait mirrors the C function signatures. JNA maps Scala strings to C character pointers and JNA Pointer values to native pointers. Once the library is loaded with `Native.load`, the exported C functions can be called as ordinary Scala methods.

The call to `gpu_evaluate_rule` is synchronous. The Scala thread blocks while the CUDA kernels execute. When the function returns, the result buffer already contains `support`, `body size`, `confidence`, and `timing` values. No sockets, JSON communication, callbacks, or background polling are required in the GPU path.

3.4 Data Flow and Output Layout

The graph is loaded once at startup. Individual rules are then evaluated one by one. The output buffer has a fixed binary layout. The native side writes integers and doubles at known byte offsets, and Scala reads them using JNA Memory methods.

Offset	Field	Type
0-3	support	int
4-7	headSize	int
8-11	headSupport	int
16-23	headCoverage	double
24-27	bodySize	int
32-39	confidence	double
40-47	supportMs	double
48-55	confidenceMs	double

Table 2 - Binary output layout returned by the native GPU backend.

Using a raw memory buffer avoids problems with mapping C structs to Scala classes and removes unnecessary serialization overhead. The only serialized input is the rule itself, passed either as a text rule or as a JSON rule object.

3.5 CPU Fallback Path

The integration also supports a CPU fallback. If no compatible GPU is available, or if the shared library cannot be loaded, Scala can launch the optimized CPU binary as a subprocess with the `--json` flag. The CPU program writes a JSON array to standard output and timing information to standard error. Scala parses the JSON and constructs the same result objects as in the GPU path.

This fallback makes the integration portable. On GPU-equipped machines, the CUDA backend provides acceleration. On CPU-only machines, RDRules can still use the native optimized implementation without requiring CUDA.

4 Experimental Evaluation

This chapter evaluates the implementations across three datasets with different structural properties. The evaluation focuses on support and confidence computation, because this is the part of the RDFRules pipeline targeted by the native and GPU implementations. Unless stated otherwise, benchmarks were run on VSE JupyterHub with an NVIDIA H200 NVL GPU. Where repeated full benchmark runs were feasible, repeated measurements were used; for expensive configurations, one full run was reported. Per-rule averages and medians describe the distribution across evaluated rules.

Two timing concepts are distinguished throughout the evaluation. Wall time includes graph loading, index construction, CUDA initialization, CSR upload, parsing, and rule evaluation. Rule-computation time is the sum of support and confidence kernel or function times and excludes one-time setup costs. This distinction is important because the GPU pays a larger one-time setup cost, but this cost becomes negligible when many rules are evaluated on the same graph.

4.1 Experimental Setup and Benchmark Methodology

All benchmarks were executed in the VSE JupyterHub GPU environment on the machine `gpu5`. The environment was documented in order to make the benchmark results reproducible and to clarify the hardware context in which the CPU and GPU implementations were evaluated. The machine ran Ubuntu 24.04.4 LTS with Linux kernel 6.8.0-110-generic.

The benchmark machine was equipped with two Intel Xeon 6730P CPU sockets. Each socket contained 32 physical cores, resulting in 64 physical CPU cores in total. Hyper-threading was enabled, with two hardware threads per core, giving 128 logical CPU threads. The maximum reported CPU frequency was 3.8 GHz. The CPU cache hierarchy consisted of 3 MiB of L1 data cache across 64 instances, 4 MiB of L1 instruction cache across 64 instances, 128 MiB of L2 cache across 64 instances, and 576 MiB of L3 cache across two instances. The machine had approximately 1 TiB of system RAM, with approximately 968 GiB available at the time of environment inspection.

The GPU environment consisted of four NVIDIA H200 NVL GPUs. Each GPU had 143,771 MiB of available device memory according to `nvidia-smi`. The GPUs were connected through NVLink, with the topology report showing NV6 links between the GPU pairs. The NVIDIA driver version was 580.126.20, and `nvidia-smi` reported CUDA version 13.0. The CUDA compiler available in the environment was NVCC from CUDA Toolkit

12.0, release 12.0.140. The native C++ implementations were compiled using GCC/G++ 13.3.0. Python 3.12.6 was used for environment inspection and auxiliary processing scripts.

Although the machine contained four GPUs, the CUDA implementation evaluated in this thesis uses a single-GPU execution model. The graph CSR index is uploaded to one selected GPU, and support and confidence kernels are executed on that device. Multi-GPU execution and graph partitioning were not implemented in this thesis and are left as possible future work. Therefore, the reported GPU memory usage in the benchmark tables should be interpreted as memory usage for one GPU execution context, not as combined memory across all four GPUs.

Component	Specification
Machine / host	gpu5
Operating system	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-110-generic
CPU	Intel Xeon 6730P
CPU sockets	2
Physical CPU cores	64 total, 32 per socket
Hardware threads	128 total, 2 threads per core
Maximum CPU frequency	3.8 GHz
L1 data cache	3 MiB across 64 instances
L1 instruction cache	4 MiB across 64 instances
L2 cache	128 MiB across 64 instances
L3 cache	576 MiB across 2 instances
System RAM	approximately 1 TiB
Available RAM during inspection	approximately 968 GiB
GPU	4 x NVIDIA H200 NVL
GPU memory	143,771 MiB per GPU
GPU interconnect	NVLink, NV6 topology between GPUs
NVIDIA driver	580.126.20
CUDA version reported by nvidia-smi	13.0
CUDA compiler	NVCC, CUDA Toolkit 12.0, release 12.0.140
C++ compiler	GCC/G++ 13.3.0
Python	Python 3.12.6

Table 3 - Benchmark machine hardware specification.

Two timing categories are distinguished throughout the evaluation. Wall time includes graph loading, index construction, CUDA initialization, CSR upload, parsing, and rule evaluation. Rule-computation time includes only the measured support and confidence computation itself. For the GPU implementation, support and confidence times were measured around the corresponding CUDA computation and GPU-side aggregation steps. For the CPU implementations, timings were measured around the corresponding support and confidence evaluation functions.

Unless stated otherwise, the benchmark tables report the measured runtime for the evaluated rule set on the described benchmark machine. Where per-rule averages and

medians are shown, they describe the distribution of runtimes across the evaluated rules within the benchmark, not necessarily variation across repeated full benchmark executions. This distinction is important because rule evaluation times are highly skewed: most rules finish quickly, while a small number of high-fanout rules can dominate the maximum runtime. If repeated full benchmark executions are available for a table, the number of runs and the standard deviation should be reported explicitly. If a benchmark was executed only once because of high computational cost, this limitation should be stated directly in the corresponding table or paragraph.

The number of repeated benchmark executions depended on the computational cost of each benchmark configuration. Configurations that were fast enough to repeat were executed three times, while the most expensive configurations were executed once. Specifically, the synthetic benchmark was repeated three times for Rewritten C++, Optimized CPU, GPU CUDA on 100k triples, and GPU CUDA on the full 1M-triple dataset. In the DBpedia benchmark, Optimized CPU and GPU CUDA were repeated three times, while Rewritten C++ was executed once because of its higher runtime. In the biomedical original_train benchmark, all three implementations were executed once on the 50-rule safe subset because the full runs were computationally expensive.

The reported average and median values in the per-rule tables are calculated across all evaluated rules within the given benchmark run or runs. Therefore, these values describe the distribution of rule evaluation times across the benchmarked rule set. Where three repeated executions were available, the final reported values were calculated from the repeated measurements; where only one execution was feasible, the results should be interpreted as descriptive measurements for that benchmark configuration rather than statistically stable estimates across repeated full runs.

4.2 Datasets and Rule Sets

Dataset	Triples	Predicates	Unique terms	Rules evaluated	Purpose
Synthetic gpu-test-100k	100,000	1	17,527	10	Dense stress test for GPU parallelism
DBpedia	~1.7 million	dozens	-	10,000	Real-world heterogeneous graph
Biomedical / original_train	6,794,977	26	4,322,470	50-rule sample	Large real-world biomedical graph

Table 4 - Datasets used in the experimental evaluation.

The synthetic dataset contains one predicate, `interacts_with`, and was generated specifically to create dense adjacency lists and large body sizes. It represents a best-case scenario for GPU parallelism.

The DBpedia dataset is a heterogeneous encyclopaedic graph with many predicates such as `birthPlace`, `country`, `capital`, `influencedBy`, and related relations. The 10,000 rules used in this benchmark were mined by RDFSRules and therefore represent realistic mining output.

The biomedical dataset is the largest benchmark. It is a Biolink-based knowledge graph with 6.8 million triples and 26 relation types. The 50-rule sample excludes rules containing `rdf:type` in the body, because `rdf:type` causes near-billion-pair body sizes that are not computable on a single machine in reasonable time.

Before measuring the Scala RDFSRules baseline, a separate JVM warm-up phase was executed. The warm-up consisted of evaluating one throwaway rule before the timed benchmark started. This rule was not included in the measured support-counting time. The purpose of this step was to trigger JVM initialization effects, JIT compilation of frequently used code paths, and especially materialisation of RDFSRules lazy index structures. RDFSRules constructs its TripleIndex lazily, meaning that some internal data structures are built only on first access rather than during initial graph loading.

In the Scala benchmark, this warm-up phase took 217.480 seconds and was recorded separately from the measured support-counting time of 417.448 seconds. Therefore, the reported Scala support-counting result does not include the initial warm-up cost. However, the warm-up time is still relevant for interpreting end-to-end execution, because it includes the first-access cost of lazy index construction and JVM runtime stabilization.

4.3 Scala Baseline versus Faithful C++ Rewrite

The first comparison isolates the effect of moving from Scala/JVM to native C++ while keeping the same basic algorithm and data structures. The benchmark uses 931 rules from the biomedical graph and measures support counting. Both implementations produced identical support values.

Implementation	Total time	Average per rule	Median per rule	Maximum per rule	Speedup
Scala RDFSRules	417.4 s	448.4 ms	184.3 ms	17,500 ms	1.0x
Rewritten C++	201.0 s	215.3 ms	97.0 ms	6,916 ms	2.1x

Table 5 - Support counting on the biomedical graph, 931 rules.

The faithful C++ rewrite is approximately 2.1 times faster than Scala. Since the algorithm and data layout are intentionally similar, this speedup is mainly caused by eliminating JVM overhead such as object allocation, garbage collection, boxed values, and JIT variability.

The result confirms that native implementation alone improves performance, but it does not remove the fundamental memory-access bottleneck.

4.4 Synthetic Stress Test: gpu-test-100k

The synthetic dataset evaluates the implementations under a dense single-predicate workload. Ten rules were evaluated, ranging from one-atom rules to three-atom joins producing millions of body pairs. The Scala RDRules baseline was measured on the canonical Rule 7 only, where confidence computation took 744,411 ms.

Implementation	Wall time	Speedup vs Scala Rule 7	Speedup vs Rewritten
Scala RDRules (Rule 7 only)	744,411 ms	1.0x	-
Rewritten C++	106,685 ms	7.0x	1.0x
Optimized CPU	60,581 ms	12.3x	1.76x
GPU CUDA	2,450 ms	303.8x	43.5x

Table 6 - Wall times on the synthetic 100k dataset.

Implementation	Rule-computation time	Speedup vs Rewritten
Rewritten C++	106,785 ms	1.0x
Optimized CPU	60,498 ms	1.77x
GPU CUDA	301 ms	354.8x

Table 7 - Rule-computation time on the synthetic 100k dataset.

The result shows the best-case behavior for GPU acceleration. Confidence counting dominates both CPU implementations, exceeding 99% of total computation time. The optimized CPU implementation improves performance by 1.77 times due to CSR locality. The GPU reduces rule-computation time from more than 100 seconds to approximately 301 ms, giving a 354.8x speedup over the faithful C++ rewrite.

Rule	Body size	Rewritten confidence	Optimized CPU confidence	GPU confidence
1	100,000	21 ms	5.3 ms	0.065 ms
3	584,395	128 ms	52.1 ms	1.387 ms
5	71,711,084	28,016 ms	5,905.0 ms	2.286 ms
7	3,711,303	60,632 ms	26,707.0 ms	86.019 ms
9	3,711,303	10,899 ms	26,656.4 ms	59.361 ms

Table 8 - Selected per-rule confidence times on the synthetic dataset.

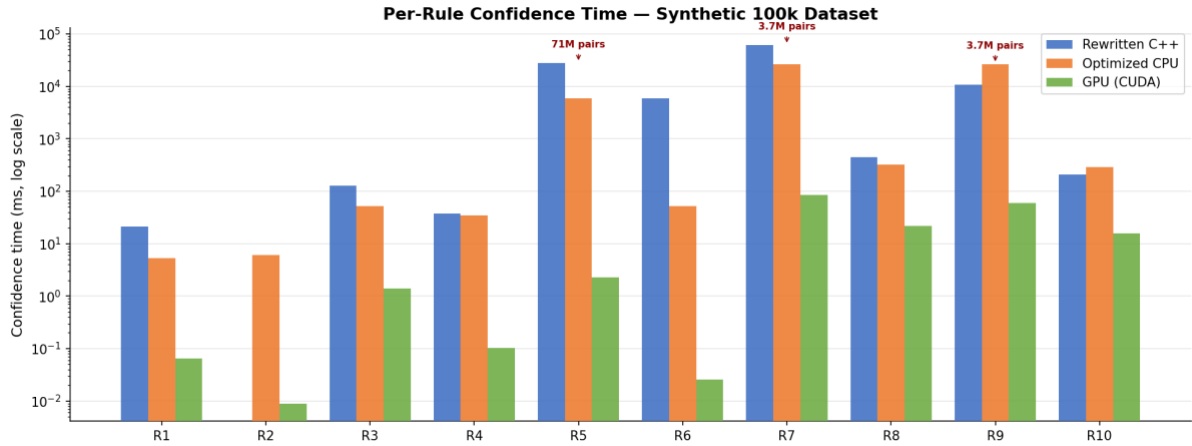


Figure 1 — Per-rule confidence time on the synthetic 100k dataset (log scale). Rule 5 with 71M body pairs shows over 12,000x GPU advantage over Rewritten C++.

The same rule set was also evaluated on a 1M-triple version of the synthetic graph. Total GPU time increased from approximately 2,211 ms to 19,691 ms, an 8.9x increase for a 10x larger graph. This indicates near-linear scaling, with constant overheads such as CSR upload partially amortized.

4.5 DBpedia Dataset: 10,000 Rules

The DBpedia benchmark evaluates aggregate throughput on a realistic heterogeneous graph. The workload consists of 10,000 rules: 164 one-atom rules, 1,240 two-atom rules, and 8,596 three-atom rules. Unlike the synthetic graph, most DBpedia predicates have smaller adjacency lists and body sizes are usually moderate.

Implementation	Wall time	Wall speedup	Rule compute	Compute speedup
Rewritten C++	57,508 ms	1.0x	53,148 ms	1.0x
Optimized CPU	43,496 ms	1.32x	43,079 ms	1.23x
GPU CUDA	6,082 ms	9.46x	2,699 ms	19.69x

Table 9 - DBpedia benchmark results.

Implementation	Support time	Confidence time
Rewritten C++	2,696 ms	50,452 ms
Optimized CPU	3,305 ms	39,729 ms
GPU CUDA	659 ms	2,741 ms

Table 10 - DBpedia support and confidence phase breakdown.

The GPU achieves a 19.69x rule-computation speedup and a 9.46x wall-time speedup. The difference between these two values is caused by the one-time index build and CSR upload cost, which is included in wall time. Because this benchmark evaluates 10,000 rules, the setup cost is already reasonably amortized.

The optimized CPU implementation is only 1.23x faster than Rewritten C++ on rule computation. This is much lower than on the synthetic dataset because DBpedia has many predicates with short or medium adjacency lists. The CSR layout therefore provides less benefit than on a dense single-predicate graph.

Body atoms	Rules	Rewritten avg confidence	Optimized CPU avg confidence	GPU avg confidence	GPU median confidence
1	164	0.0 ms	0.2 ms	0.090 ms	0.086 ms
2	1,240	7.7 ms	2.0 ms	0.128 ms	0.094 ms
3	8,596	4.8 ms	4.3 ms	0.260 ms	0.103 ms

Table 11 - DBpedia confidence times by body-atom count.

The GPU median confidence time remains almost flat across body sizes, increasing only from 0.086 ms for one-atom rules to 0.103 ms for three-atom rules. This suggests that GPU execution is dominated more by available parallel work and launch overhead than by rule depth for most DBpedia rules.

4.6 Biomedical Dataset: original_train

The biomedical benchmark is the largest and most irregular dataset. It contains 6,794,977 triples, 4,322,470 unique terms, and 26 predicates. The evaluated 50-rule sample excludes rdf:type body atoms because they generate excessively large body sizes. This benchmark is important because it demonstrates that CPU memory-layout optimization is not universally beneficial.

Implementation	Wall time	Wall speedup vs Rewritten	Rule compute	Compute speedup vs Rewritten	Peak memory
Rewritten C++	38,136 ms	1.0x	20,297 ms	1.0x	2,112 MB RAM
Optimized CPU	104,560 ms	0.37x	95,312 ms	0.21x	1,430 MB RAM
GPU CUDA	13,639 ms	2.80x	1,202 ms	16.89x	1,692 MB RAM + 9,733 MB VRAM

Table 12 - Biomedical 50-rule benchmark results.

The GPU achieves a 16.89x rule-computation speedup over Rewritten C++ and a 2.80x wall-time speedup. The smaller wall-time speedup is caused by the one-time GPU setup cost: graph indexing and CSR upload take approximately 9.4 seconds. Since only 50 rules are evaluated, this setup cost forms a large part of total wall time. For hundreds or thousands of rules, the same cost would be amortized more effectively.

The optimized CPU implementation is slower than the faithful C++ rewrite on this dataset. This is caused by a small number of high-fanout outlier rules where the CSR traversal enumerates very large candidate ranges. The hash-map based DFS in Rewritten C++ can prune some of these cases earlier, while the CSR implementation materializes or visits many candidates before deduplication.

Rule	Body size	Body pattern summary	Rewritten confidence	Optimized CPU confidence	GPU confidence
22	3,364,149	consumes / assesses / consumes	16,062 ms	36,177 ms	992 ms
23	2,390,984	capable_of / assesses / consumes	2,505 ms	6,345 ms	75 ms
50	193,954	subclass_of / has_chemical_role / subclass_of	50 ms	50,975 ms	5 ms

Table 13 - Outlier rules on the biomedical dataset.

Rule 50 is the most important anomaly. Its body size is moderate compared to Rules 22 and 23, but the optimized CPU implementation is approximately 1,020 times slower than Rewritten C++ for confidence counting. The reason is the high fanout of the subclass_of predicate. CSR enumeration produces a massive intermediate search space, while the hash-map DFS avoids much of this work through pruning. This result is important because it shows that cache-friendly layout alone is not sufficient; traversal strategy and graph topology also determine performance.

Body atoms	Rules	Rewritten avg confidence	Optimized CPU avg confidence	GPU avg confidence	GPU max confidence
1	9	25.4 ms	7.5 ms	0.445 ms	0.621 ms
2	19	38.9 ms	65.3 ms	1.004 ms	4.739 ms
3	22	865.6 ms	4,267.0 ms	53.2 ms	991.99 ms

Table 14 - Biomedical confidence time by body-atom count.

4.7 Memory Usage

Implementation	Synthetic 100k RAM	DBpedia RAM	Biomedical RAM	Biomedical VRAM
Rewritten C++	3,082 MB	120 MB	2,112 MB	-
Optimized CPU	16 MB	113 MB	1,430 MB	-
GPU CUDA	258 MB	365 MB	1,692 MB	9,733 MB

Table 15 - Peak memory usage across datasets.

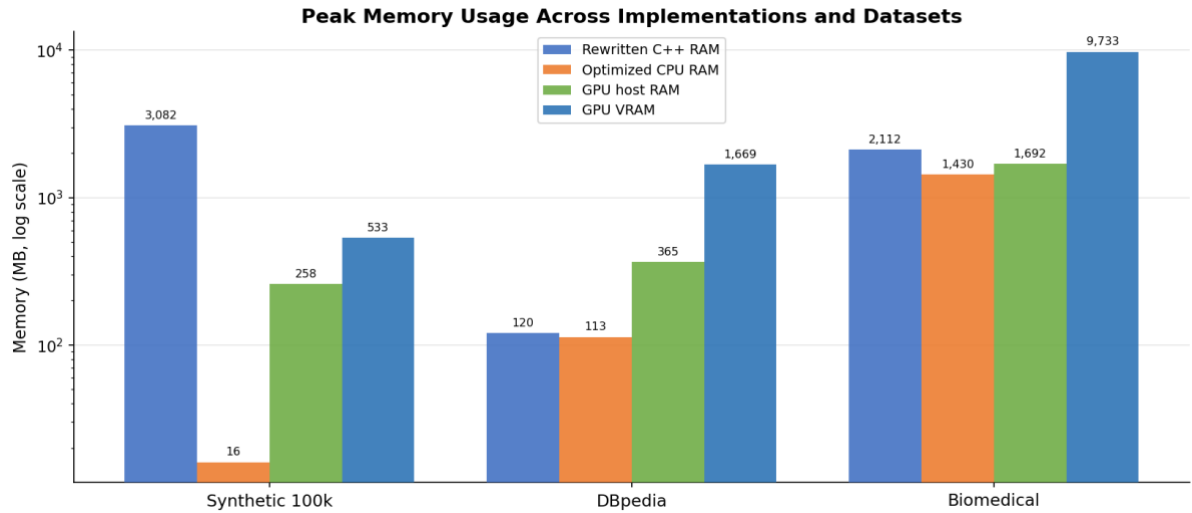


Figure 2 — Peak memory usage (log scale).

Memory usage shows another trade-off. The hash-map based Rewritten C++ implementation can use large amounts of RAM due to pointer overhead, especially on the synthetic dataset. The optimized CPU CSR representation is much more compact in RAM. The GPU implementation requires both host memory and VRAM because the complete CSR index is stored on the GPU. On the biomedical dataset, this requires approximately 9.7 GB of VRAM.

4.8 Cross-Dataset Discussion

Dataset	Rewritten rule compute	Optimized CPU vs Rewritten	GPU vs Rewritten	GPU VRAM
Synthetic 100k	106,785 ms	1.77x faster	354.8x faster	533 MB
DBpedia	53,148 ms	1.23x faster	19.69x faster	1,669 MB
Biomedical	20,297 ms	4.7x slower	16.89x faster	9,733 MB

Table 16 - Cross-dataset rule-computation summary.

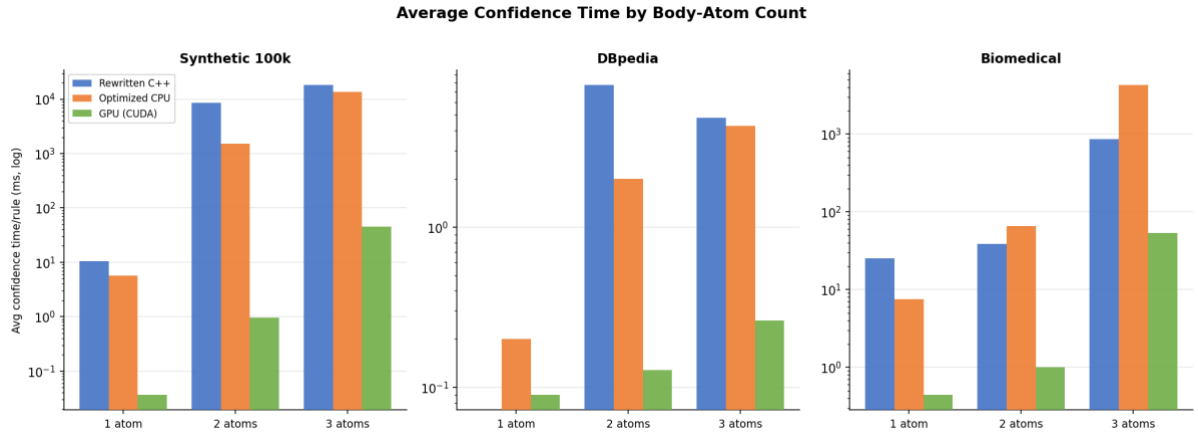


Figure 3 — Average confidence time by body-atom count across all three datasets (log scale). Note the CPU regression on 3-atom biomedical rules caused by outlier rules 22, 23, 50.

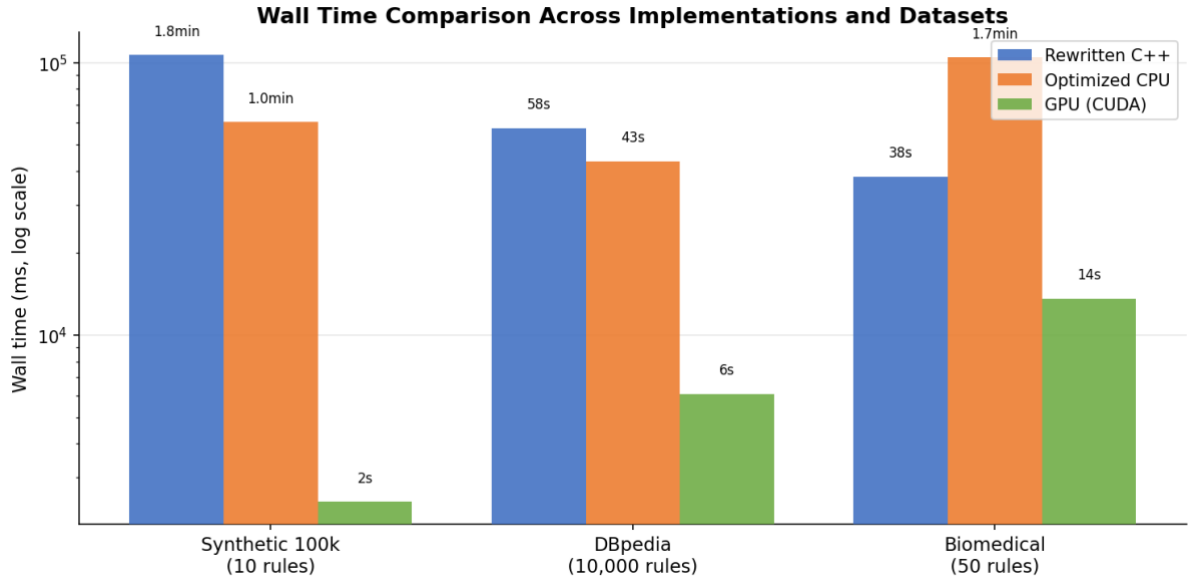


Figure 4 — Wall time comparison (log scale) across all datasets. GPU wall times include the one-time index build and CSR upload cost.

Several findings are consistent across the experiments. First, confidence counting dominates CPU execution. Support counting is comparatively cheap because it stops after the first satisfying body grounding for each head pair. Confidence counting must enumerate and deduplicate all body-induced pairs, which makes it the natural target for GPU acceleration.

Second, GPU advantage increases with available parallelism. On the dense synthetic graph, the GPU achieves more than 350x rule-computation speedup because body-size enumeration produces millions of independent pairs. On DBpedia and the biomedical graph, the speedup is lower but still substantial: approximately 20x and 17x respectively on rule computation.

Third, CPU CSR optimization is topology-dependent. It is useful on dense uniform graphs and modestly useful on DBpedia, but it can be much slower on high-fanout biomedical rules. Therefore, the optimized CPU implementation should not be presented as universally better than the faithful C++ rewrite. It is a useful alternative with lower memory usage and good performance on suitable graphs, but it has pathological cases.

Fourth, GPU setup cost must be interpreted correctly. The GPU has to parse the graph, build CSR indexes, upload data to VRAM, and initialize CUDA resources. This one-time cost is visible in small workloads such as the 50-rule biomedical sample. However, in realistic mining settings where hundreds or thousands of rules are evaluated on the same graph, the setup cost is amortized and rule-computation speedup becomes the more relevant metric.

4.9 Implementation Selection Guidelines

Scenario	Recommended implementation	Reason
Many rules on a graph that fits in GPU memory	GPU CUDA	Index cost is amortized and compute speedup is large.
Small rule set without GPU	Rewritten C++	No GPU setup cost and predictable DFS pruning.
Dense uniform graph without GPU	Optimized CPU	CSR locality provides clear benefit.
High-fanout predicates with few rules	Rewritten C++ or GPU	CSR CPU may regress due to excessive enumeration.
Memory-constrained CPU environment	Optimized CPU	CSR representation uses less RAM than hash maps.
Graph too large for available VRAM	CPU fallback	GPU requires the full CSR index in device memory.

Table 17 - Recommended implementation by workload scenario.

5 Conclusion

This thesis investigated the acceleration of support and confidence computation in inductive logic rule mining over RDF-style knowledge graphs. The work focused on the counting phase of an AMIE-style rule mining workflow, because this phase repeatedly evaluates join-like graph patterns over large triple sets and becomes one of the main computational bottlenecks in practical rule mining systems.

The main contributions of the thesis are three implementations of the same counting semantics. First, a faithful C++ rewrite of the RDRules counting logic was created to preserve support, body-size, confidence, injective mapping, and DFS-based evaluation while removing Scala/JVM overhead. Second, an optimized non-parallel CPU version was implemented using CSR-style indexes, stack-local DFS state, bitmask atom tracking, and body-size-specialized DFS functions. This improved performance on dense and regular graphs, but also showed that CSR traversal can regress on high-fanout predicates where the original hash-map DFS prunes earlier.

The main contribution is the CUDA GPU implementation. The final GPU backend stores predicate indexes in GPU memory, evaluates support with massively parallel DFS kernels, and computes body size and confidence using GPU-side pair collection, bitmap counting, or Thrust-based deduplication. The evaluation showed strong rule-computation speedups: 354.8x on the synthetic dataset, 19.69x on DBpedia, and 16.89x on the biomedical dataset compared with the faithful C++ rewrite. These results confirm that GPU acceleration is most effective when confidence computation creates a large amount of independent parallel work and when setup costs can be amortized across many rules.

The unsuccessful GPU attempts were also important. The binary-search GPU version showed that parallel DFS alone is not enough if predicate lookups remain expensive. The sparse-matrix approach showed that SpGEMM-based evaluation is not feasible for the tested graphs because intermediate matrices can become too large. These findings motivated the final CSR-indexed DFS design.

Several limitations remain. The GPU backend currently accelerates metric computation, but it is not yet fully embedded inside the RDRules rule generation loop. Future work should focus on tighter Scala/RDRules integration, where the native GPU evaluator becomes a standard backend used directly during mining. A deeper integration could allow candidate rules to be batched, ordered by expected cost, and evaluated on the GPU during rule generation, so pruning decisions could immediately use GPU-computed support and confidence values.

Future research could also explore adaptive hybrid execution, where each rule is routed to the most suitable backend: faithful C++, optimized CPU CSR, or GPU CUDA. Multi-GPU execution and graph partitioning are also relevant for datasets that exceed single-GPU memory capacity. Overall, the thesis shows that GPU acceleration is not universally superior, but it is highly effective for the data-parallel confidence-counting workloads that dominate large-scale RDF rule evaluation.

Sources

Bordes, A., Usunier, N., García-Durán, A., Weston, J. and Yakhnenko, O. (2013) 'Translating Embeddings for Modeling Multi-relational Data', in *Advances in Neural Information Processing Systems 26 (NIPS 2013)*. Available at: <https://proceedings.neurips.cc/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html> (Accessed: 1 May 2026).

Chen, Y., Goldberg, S., Wang, D.Z. and Johri, S.S. (2016) 'Ontological Pathfinding', in *Proceedings of the 2016 ACM International Conference on Management of Data (SIGMOD '16)*. New York: ACM, pp. 835–846. Available at: <https://doi.org/10.1145/2882903.2882954> (Accessed: 1 May 2026).

Cheng, J., Grossman, M. and McKercher, T. (2014) *Professional CUDA C Programming*. Indianapolis, IN: Wrox.

Galárraga, L., Teflioudi, C., Hose, K. and Suchanek, F.M. (2015) 'Fast rule mining in ontological knowledge bases with AMIE+', *The VLDB Journal*, 24(6), pp. 707–730. Available at: <https://luisgalarraga.de/docs/amie-plus.pdf> (Accessed: 2 February 2026).

Hennessy, J.L. and Patterson, D.A. (2012) *Computer Architecture: A Quantitative Approach*. 5th edn. Amsterdam: Elsevier / Morgan Kaufmann.

Kirk, D.B. and Hwu, W.-m.W. (2013) *Programming Massively Parallel Processors: A Hands-on Approach*. 2nd edn. Amsterdam: Morgan Kaufmann.

Kirk, D.B. and Hwu, W.-m.W. (2016) *Programming Massively Parallel Processors: A Hands-on Approach*. 3rd edn. Amsterdam: Morgan Kaufmann.

Lajus, J., Galárraga, L. and Suchanek, F.M. (2020) 'Fast and Exact Rule Mining with AMIE 3', in *Proceedings of the Extended Semantic Web Conference (ESWC 2020)*, pp. 36–52. Available at: <https://luisgalarraga.de/docs/amie3.pdf> (Accessed: 1 May 2026).

Meilicke, C., Chekol, M.W., Ruffinelli, D. and Stuckenschmidt, H. (2019) 'Anytime Bottom-Up Rule Learning for Knowledge Graph Completion', in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*. Macao: IJCAI, pp. 3137–3143. Available at: <https://web.informatik.uni-mannheim.de/AnyBURL/meilicke19anyburl.pdf> (Accessed: 1 May 2026).

NVIDIA Corporation (n.d.) *CUDA Toolkit Documentation*. Available at: <https://docs.nvidia.com/cuda/> (Accessed: 30 April 2026).

NVIDIA Corporation (n.d.) Thrust Documentation. Available at: <https://nvidia.github.io/cccl/thrust/> (Accessed: 30 April 2026).

NVIDIA Corporation (n.d.) cuSPARSE Documentation. Available at: <https://docs.nvidia.com/cuda/cusparse/> (Accessed: 30 April 2026).

Sun, Z., Deng, Z., Nie, J.Y. and Tang, J. (2019) 'RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space', in *Proceedings of the 7th International Conference on Learning Representations (ICLR 2019)*. Available at: <https://openreview.net/forum?id=HkgEQnRqYQ> (Accessed: 1 May 2026).

Zeman, V., Kliegr, T. and Svátek, V. (2021) RDFRules: Making RDF rule mining easier and even more efficient. *Semantic Web*, 12(4), pp. 569-602. Available at: <https://www.semantic-web-journal.net/system/files/swj2398.pdf> (Accessed: 2 February 2026).

Schema Sources

Schema 1 — CPU Architecture Diagram: Red Hat, Inc. (n.d.) CPU components and their functionality. Available at: <https://www.redhat.com/en/blog/cpu-components-functionality> (Accessed: 24 January 2026).

Schema 2 — GA100 GPU with 128 SMs: Krashinsky, R., Giroux, O., Jones, S., Stam, N. and Ramaswamy, S. (2020) NVIDIA Ampere Architecture In-Depth. NVIDIA Developer Blog, 14 May. Available at: <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/> (Accessed: 24 January 2026).

Schema 3 — GA100 SM Architecture: Krashinsky, R., Giroux, O., Jones, S., Stam, N. and Ramaswamy, S. (2020) NVIDIA Ampere Architecture In-Depth. NVIDIA Developer Blog, 14 May. Available at: <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/> (Accessed: 24 January 2026).

Schema 4 — Kernel execution on GPU: NVIDIA Corporation (2020) CUDA refresher: The CUDA programming model. Available at: <https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/> (Accessed: 27 January 2026).

Schema 5 — Memory hierarchy on GPUs: NVIDIA Corporation (2020) CUDA C++ Programming Guide: Memory Hierarchy. Available at: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#memory-hierarchy> (Accessed: 2 February 2026).

Schema 6 — RDFRules rule mining workflow: computational bottlenecks on CPU: own processing.

Schema 7 — CPU implementation: own processing.

Schema 8 — GPU Attempt 1: parallel DFS with binary-search lookup: own processing.

Schema 9 — GPU Attempt 2: SpGEMM approach and CSR discovery: own processing.

Schema 10 — GPU_03 final design: data flow and execution: own processing.

Schema 11 — Body-size counting: bitmap and retry-buffer strategies: own processing.

Schema 12 — GPU_03 CSR-indexed DFS: step-by-step support and confidence counting: own processing.

Schema 13 — Full integration architecture— Scala to native GPU: own processing.

Appendix

Two artificial intelligence tools were used during the preparation of this master thesis: Claude (Anthropic) and ChatGPT (OpenAI). In the written part of the thesis, these tools were used for improving grammar and language style of the English text, for reformulating and expanding individual text passages, and for brainstorming during the formulation of research goals. In all cases, the factual content, expert conclusions, interpretation of results, and research findings are exclusively the author's own contribution.

In the practical part, AI tools were used to support the implementation of the C++ and CUDA source code. All key technical decisions were proposed and directed by the author, including the CSR graph layout, the DFS-based parallelisation strategy, the bitmap and retry-buffer approaches for body-size counting, the warp-cooperative kernel design, and the JNA integration architecture. Thanks to AI assistance, the author was able to translate these ideas into working implementation significantly faster than would otherwise have been possible. AI tools helped generate boilerplate code structures and suggested standard CUDA and C++ patterns based on the author's specific instructions. All generated code was reviewed, tested, and validated by the author against the correctness baseline of the Scala RDFRules implementation. All benchmark measurements and interpretation of results are entirely the author's own work.

Source Code and Jupyter Notebooks

The complete list of all Jupyter notebooks and Python scripts used in this project, along with a one-line description of their purpose, is provided below. All source code is publicly available in the repository on GitHub:

https://github.com/vojtechnaar/MasterThesis_GPU_Acceleration

Rewritten C++ — Baseline Implementation

RunCodeRewritten.ipynb — Compiles and runs the faithful C++ rewrite of the Scala RDFRules support and confidence counting logic; used to establish the C++ correctness baseline.

Optimized CPU Implementation

RunCodeCPU01.ipynb — Compiles and runs the cache-optimized CSR-based CPU implementation (CPU_01_nonparallel); benchmarks support and confidence counting against the C++ baseline.

GPU CUDA Implementation

RunCodeGPU03.ipynb — Compiles the final GPU_03 CUDA implementation, uploads the CSR index to GPU VRAM, and benchmarks support and confidence counting across all datasets.

Benchmarking and Evaluation Scripts

benchmark_dbpedia.ipynb — Runs the full DBpedia 10,000-rule benchmark across all three implementations and records wall times, rule-compute times, and per-body-size statistics.

benchmark_original_train.ipynb — Runs the 50-rule biomedical dataset benchmark; captures outlier rule timing, memory usage, and per-phase breakdown.

benchmark_gpu_test_100k.ipynb — Runs the synthetic single-predicate stress test on the 100k and 1M triple graphs; measures GPU parallelism advantage at maximum load.

scala_baseline_support.ipynb — Runs the Scala RDFS rules support-counting baseline on the biomedical graph (931 rules) and records timing for comparison with the C++ rewrite.

Data Generation Scripts

tsv_to_ttl.py — Converts a TSV file of entity-relation-entity triples into Turtle (.ttl) format; used to generate the synthetic gpu-test-100k and gpu-test-1M benchmark graphs.